

VIT®
BHOPAL
www.vitbhopal.ac.in

Computer Vision

Module 1

Dr. Neha Rathore
Assistant Professor, SCAI
neharathore@vitbhopal.ac.in

1. WHAT IS AN IMAGE?

An image is simply a visual representation of an object, scene, or person.

EXAMPLES


Flower


Honey Bee


X-ray

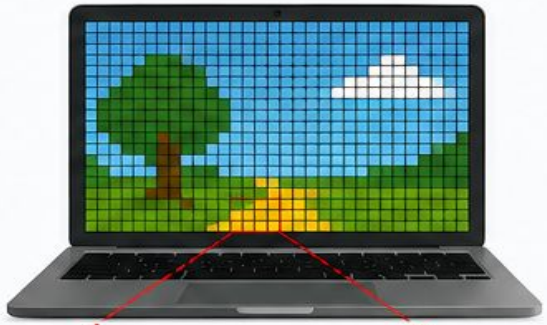

Satellite Image

2. WHAT IS A DIGITAL IMAGE?

- A digital image is an image that is stored inside a computer in the form of **numbers (binary values)**.
- A computer only understands **0 and 1**.
- A digital image is a **two-dimensional function f(x, y)** where the intensity or color at every position is represented by a numerical value.

Here,

- **x** → horizontal position
- **y** → vertical position
- **f(x, y)** → brightness or color value at that position



120	135	150	160	170
115	128	140	155	165
110	120	135	150	160
100	110	125	140	155
90	100	115	130	145

Each number represents the intensity (brightness/color) of a pixel.

3. REAL WORLD vs DIGITAL IMAGE

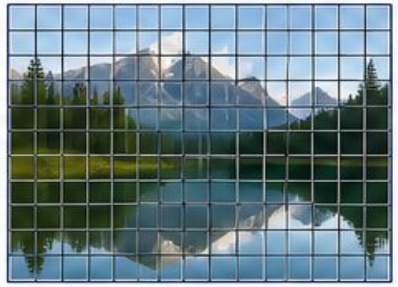
Real World


Continuous scene (what we see)

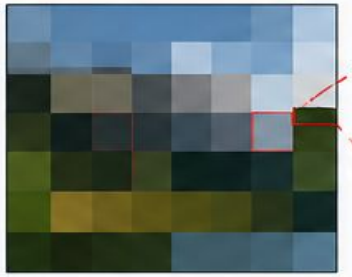
Captured by Camera


Camera captures the continuous scene

Sampling


Continuous image is divided into small squares (pixels)

Pixels (Zoomed)


Each small square is a pixel

Digital Image (Matrix)

120	135	150	160	170
115	128	140	155	165
110	120	135	150	160
100	110	125	140	155
90	100	115	130	145

Each pixel has a numerical value → Digital Image

Pixel and Pixel Grid

4.1 WHAT IS A PIXEL?

- **Pixel** stands for **Picture Element**.
- It is the smallest unit of a digital image.
- Every pixel has a **location** (row, column) and stores a **numerical value** that represents **brightness** (in gray images) or **color** (in color images).

EXAMPLE

Different pixel values represent different brightness levels.



0 Black → 255 White

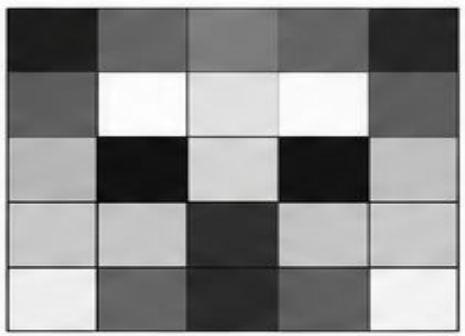
KEY POINTS

- More pixels → Better detail
- Each pixel → One numeric value (gray) or three values (R,G,B) in color images

4.2 EXAMPLE OF PIXEL GRID (IMAGE)

A digital image is divided into a grid of tiny squares (pixels). Each square stores a number.

Example: 5 × 5 Pixel Image (Grayscale)



Pixel Grid

45	60	70	90	100
50	65	80	95	105
55	75	90	100	120
60	80	95	110	130
70	90	105	120	140

Pixel Values (Intensity)

Explanation:

- The image size is 5 × 5 (5 rows and 5 columns).
- Each square is a pixel.
- The number inside each cell is the intensity of that pixel.
- Values can range from **0 (black)** to **255 (white)** in 8-bit images.

Image Representation

A digital image can be represented mathematically as a 2-D function $f(x, y)$, where (x, y) is the position of a pixel and $f(x, y)$ is its intensity (or color).

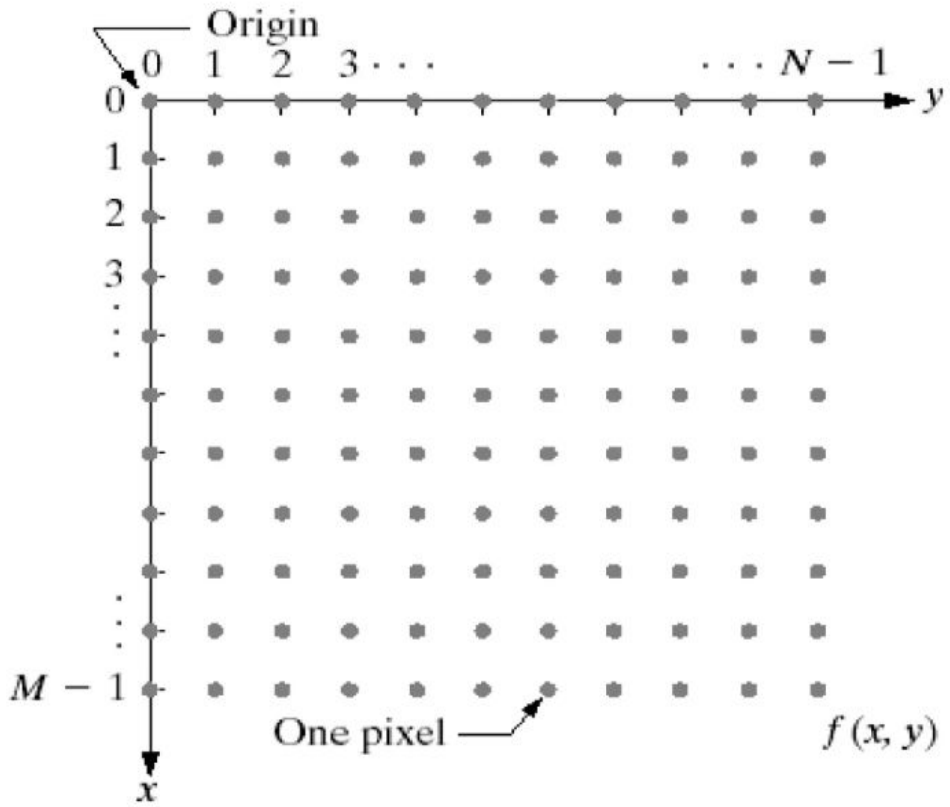
Coordinate System

$x \rightarrow$ (column index)

$y \downarrow$ (row index)	0	1	2	3
0	20	35	80	90
1	40	50	70	95
2	65	80	90	120
3	75	85	100	130

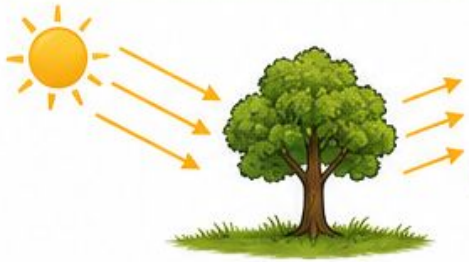
Example

At position $(x = 2, y = 1)$,
the pixel intensity $f(2, 1) = 70$



How digital image formed?

1. Light from the Scene



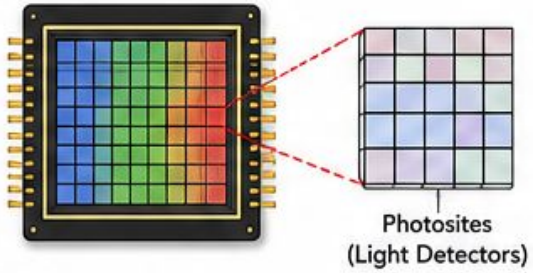
- Every object **reflects light.**
- The reflected light carries **information** about the object such as color, brightness, texture, and shape.
- This reflected light is the source for forming an image.

2. Lens Collects and Focuses Light



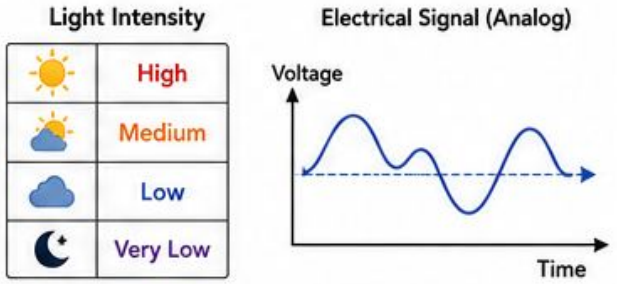
- The camera lens collects the reflected light from the scene.
- The lens **focuses** this light onto the image sensor.
- An inverted, real image is formed on the sensor.

3. Image Sensor Captures Light (CCD / CMOS)



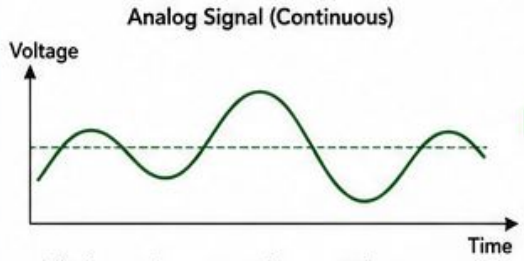
- The image sensor contains millions of tiny light detectors called **photosites.**
- Each photosite receives light.
- The amount of light falling on each photosite depends on the brightness of that point in the scene.

4. Sensor Converts Light into Electrical Signals



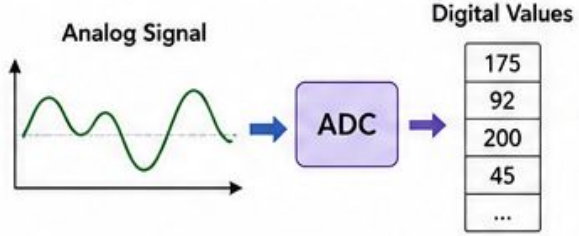
- Each photosite converts the received light into an electrical charge.
- More light → Higher electrical signal
- Less light → Lower electrical signal
- The sensor outputs an **analog signal** (continuous in nature).

5. Analog Signal is Produced



- All photosites together produce a **continuous** analog electrical signal.
- This signal represents the brightness information of the entire image.
- Computers cannot store analog signals directly.

6. Analog-to-Digital Conversion (ADC)



- The ADC (Analog-to-Digital Converter) **samples** the analog signal at discrete intervals.
- It converts each sample into a digital number **(quantization).**

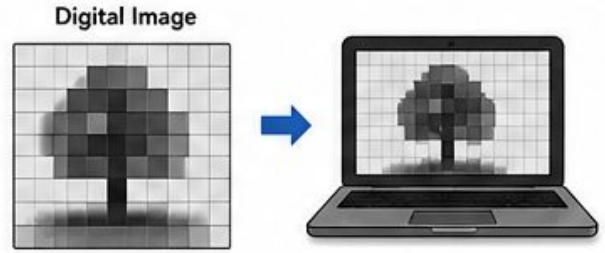
7. Digital Values (Image Matrix)

After Sampling and Quantization

120	135	150	160	170
115	128	140	155	165
110	120	135	150	160
100	110	125	140	155
90	100	115	130	145

- After ADC, each photosite has a digital value.
- These values form a **2-D array (matrix).**
- Each number represents the intensity (brightness) of a pixel.

8. Image Storage



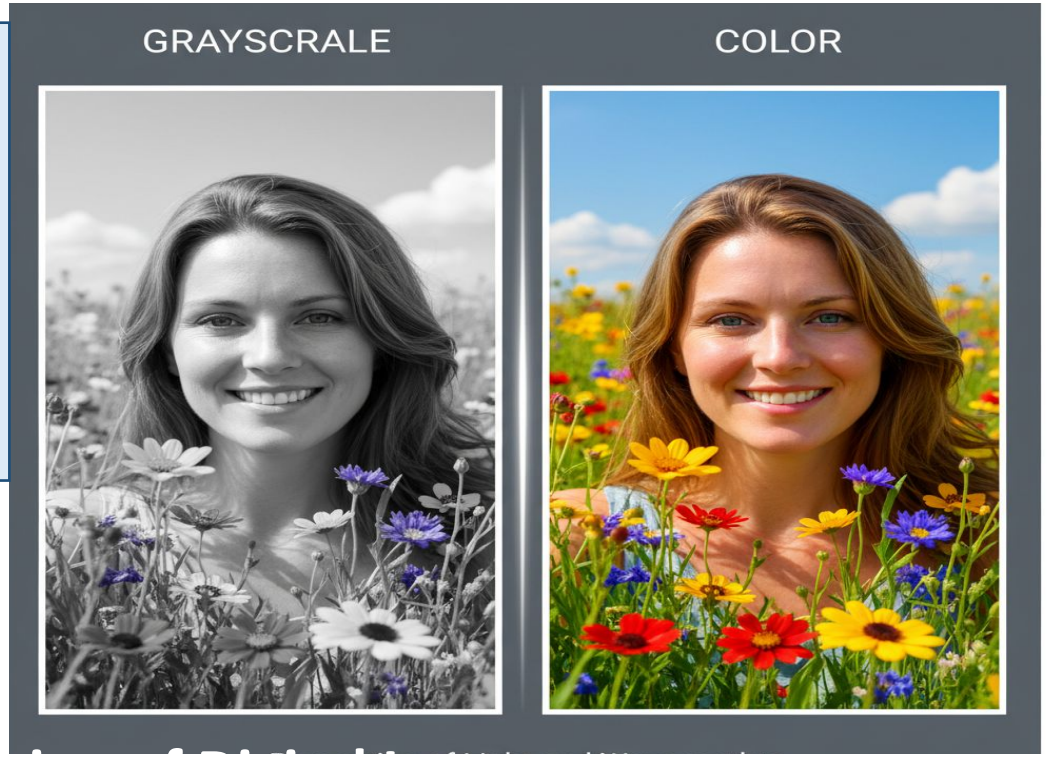
- The matrix of numbers is **stored** in the computer memory.
- The stored data can be **displayed, processed, transmitted** or saved as image files (e.g., JPG, PNG, BMP).

Gray scale and Color image

In a gray image, each pixel stores only one value.

0 ⟶ Black

255 ⟶ White



- A color image uses three primary colors.
- Red (R), Green (G), Blue (B)
- Each pixel stores three numbers.
- P(R,G,B) = (255,0,0) = Pure Red
- P(R,G,B) = (255,255,255) = white

Resolution of Digital Image: Resolution means Number of pixels present in an image

Example: 640 × 480 = 307200 pixels

Higher resolution means

- More pixels
- Better details
- Larger file size

What is Image processing?

- **Digital Image Processing (DIP)** is the process of performing various operations on a digital image using a computer to improve its quality, extract useful information, or prepare it for further analysis.

Mathematically, if

- f(x, y) represents the input image, then a digital image processing operation can be represented as

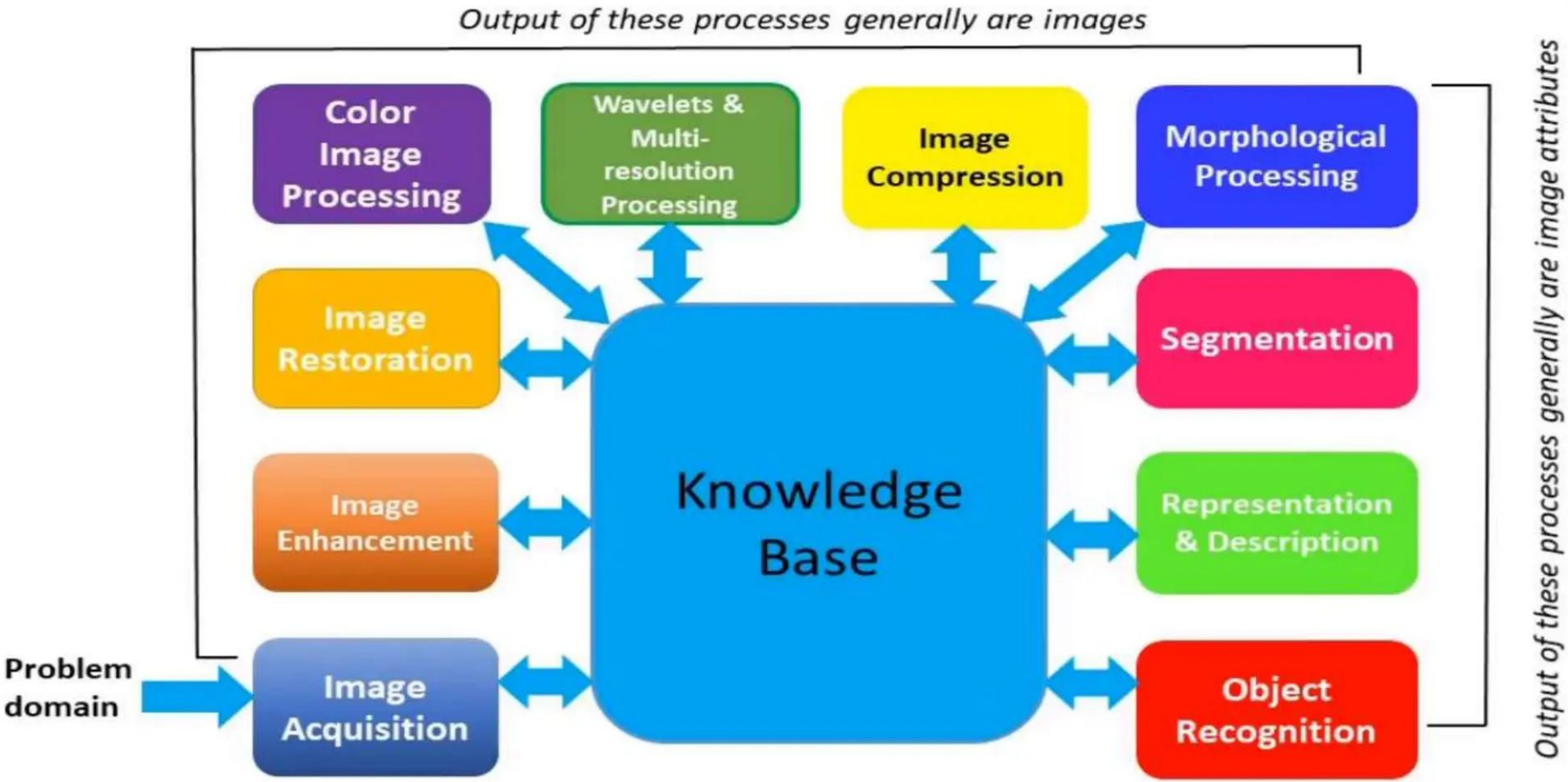
$$g(x, y)=T[f(x, y)]$$

Where, T = image processing operation

g(x,y) = processed output image

- The transformation T may represent enhancement, restoration, segmentation, filtering, compression, or feature extraction.

FUNDAMENTAL STEPS IN DIGITAL IMAGE PROCESSING



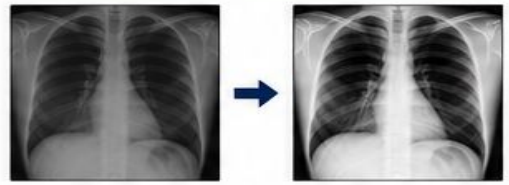
FUNDAMENTAL STEPS OF DIGITAL IMAGE PROCESSING

1. Image Acquisition



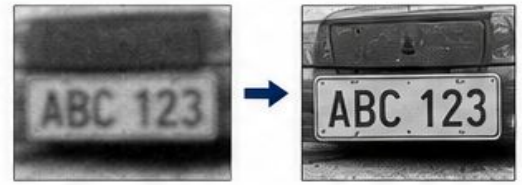
Capture image from the real world using a sensor (camera, scanner, etc.) and convert it into digital form.

2. Image Enhancement



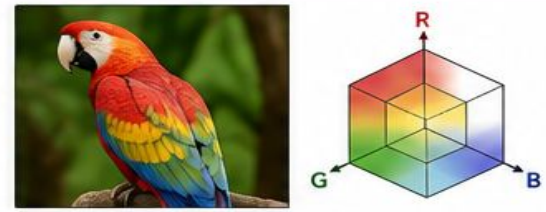
Improve the visual quality of the image (e.g., contrast stretching, histogram equalization, sharpening).

3. Image Restoration



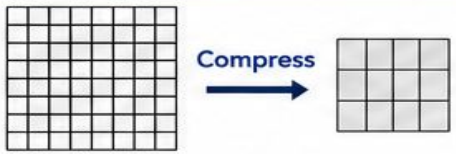
Recover the image from degradations caused by noise, blur, or other distortions using mathematical models.

4. Color Image Processing



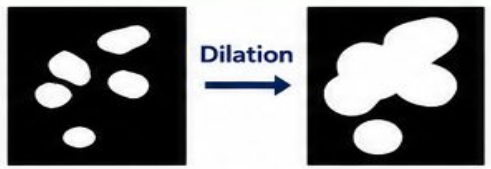
Process color images using various color models (RGB, HSV, YCbCr, Lab, etc.) for analysis and correction.

5. Image Compression



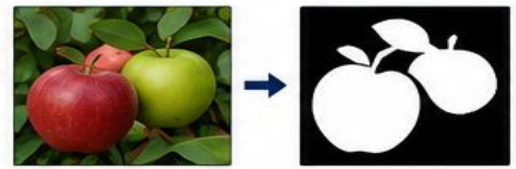
Reduce the amount of data required to store or transmit the image (e.g., JPEG, PNG, JPEG2000).

6. Morphological Processing



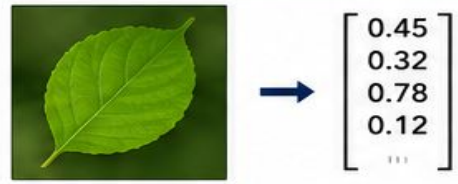
Process the structure of objects in an image using operations like erosion, dilation, opening, and closing.

7. Image Segmentation



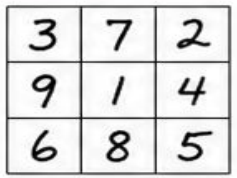
Partition the image into meaningful regions or objects (e.g., thresholding, edge detection, region growing).

8. Feature Extraction



Extract important features (shape, texture, color, edges, etc.) that represent the essential information.

9. Classification / Recognition



Classify or recognize objects in the image using machine learning or deep learning techniques.

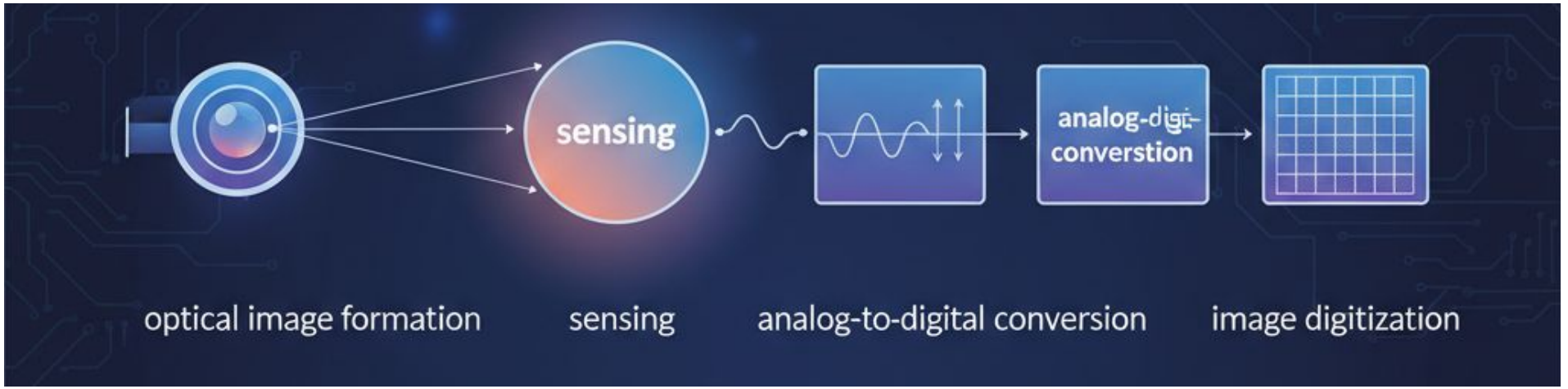
10. Decision Making



Make a final decision based on the recognition or analysis results (e.g., alert system, control system, diagnosis, automation).

Image Acquisition

- Image acquisition converts a continuous real-world scene into a digital image suitable for computational processing.



- Digital image acquisition consists of two fundamental operations:
  - **Sampling**
  - **Quantization**

Sampling and Quantization

1. SAMPLING (Spatial Sampling)

Sampling is the process of dividing a continuous image into small, discrete points called pixels.

Continuous Image (Real World)



After Sampling



How Sampling Works

- The image plane is divided into a grid of rows and columns.
- The value of the image is measured at each grid intersection.
- Each measurement point is a pixel.
- More sampling (smaller pixel size) → more pixels → better detail (higher spatial resolution).

Example: Sampling with Different Densities

Low Sampling (4 × 4) | Medium Sampling (8 × 8) | High Sampling (16 × 16)



Increasing number of samples (pixels)
Improved detail and resolution

Sampling decides HOW MANY pixels (spatial resolution).

2. QUANTIZATION (Intensity Quantization)

Quantization is the process of assigning a finite number of intensity values to the sampled pixels.

Continuous Intensity



(Infinite possible values)

After Quantization (8-bit)



(256 possible values: 0 to 255)

How Quantization Works

- Each sampled pixel has a continuous intensity value.
- This value is mapped (rounded) to the nearest available discrete intensity level.
- The number of levels is determined by the number of bits used.
- More levels (more bits) → better tonal quality (higher intensity resolution).

Example: Quantization (3-bit)

Continuous Value	Nearest Quantized Value (3-bit)	3-bit Binary Representation
12.7	13	000
45.3	45	001
102.8	103	010
150.2	150	011
200.9	201	100
230.6	231	101
250.1	250	110

8 levels (0 to 7)

Quantization decides WHAT VALUE each pixel can take (intensity/brightness resolution).

IMAGE SAMPLING & QUANTIZATION: DISCRETIZING AN IMAGE

FUNDAMENTALS OF DIGITAL IMAGES



THE IMAGE SAMPLING & QUANTIZATION PROCESS



f(x,y)

THE SAMPLING PROCESS
(Spatial Discretization)



Definition: Discretizing the spatial coordinates (x,y) of a continuous image into pixels.

Mathematically: $f(x,y) \longrightarrow f(m,n)$, where m=0...M-1 (rows), n=0...N-1 (cols)

Image represented as an M×N matrix.

THE QUANTIZATION PROCESS
(Intensity Discretization)



Definition: Mapping continuous intensity values (I_c) into a finite set of discrete gray levels (L).

Mathematically: $Q(I_c) = I_d$, where $I_d \in \{0, 1, ..., L-1\}$

L = Number of gray levels.

COMPARISON EXAMPLES

Sampling Comparison



Image A: 256×256 pixels



Image B: 2048×2048 pixels

Higher sampling frequency → more pixels, better spatial detail, larger storage.

Quantization Comparison (8-bit grayscale)



1-bit Quantization (L=2)



8-bit Quantization (L=256)

8-bit grayscale: $L=2^8=256$ gray levels (0-255). 0 = black, 255 = white.



SAMPLING specifies *where* measurements are taken. (Coordinates: m, n)



SAMPLING determines **Spatial Resolution** (ability to distinguish fine details, edges, textures).



QUANTIZATION specifies *what* numerical value is assigned to each sampled pixel. (Intensity: I_d)

Enhancement

- Improves visual quality of image.
- Brighten the image, sharpening, enhance Contrast, Equalization, Intensity adjustment etc.
  - Medical Imaging: Enhance X-rays, CT, and MRI images for accurate diagnosis.
  - Satellite Imaging: Improve visibility of roads, buildings, and vegetation.
  - Surveillance: Enhance low-light CCTV footage.
  - Autonomous Vehicles: Improve visibility under fog, rain, or low-light conditions.



Image Enhancement techniques

- Spatial domain
  - Point Processing
    - brightness adjustment
    - Contrast stretching
    - Image negative
    - Log transformation
    - Power-law (Gamma) transformation
    - Thresholding
  - Histogram-Based Techniques
    - Histogram Equalization
    - Adaptive Histogram Equalization (AHE)
    - CLAHE (Contrast Limited Adaptive Histogram Equalization)
    - Histogram Matching
  - Spatial Filtering
    - Smoothening filter
    - Sharpening filter
- Frequency domain
  - LPF
  - HPF
  - BPF
  - Wavelet filter

Image Restoration

- **Image Restoration** is the process of recovering the **original image** from a degraded image by removing or reducing distortions such as **noise, blur, motion blur, or sensor imperfections** using mathematical models.
- Images may be degraded due to:
  - Camera shake (motion blur)
  - Out-of-focus camera (defocus blur)
  - Sensor noise
  - Dust or scratches
  - Low-light conditions





Image restoration method

- Step 2: Identify the Type of Degradation

Problem	Restoration Method
Noise	Denoising
Motion Blur	Deblurring
Defocus Blur	Deconvolution
Missing Pixels	Image Inpainting

- Step 3: build Degradation model

The degraded image is mathematically modeled as:

$$g(x, y) = h(x, y) * f(x, y) + n(x, y)$$

where:

f(x,y) = Original image

h(x,y) = Blur function (Point Spread Function, PSF)

$*$ = Convolution operation

n(x,y) = Noise

g(x,y) = Degraded image

The goal is to estimate f(x,y) from the observed degraded image g(x,y).

- Step 4: Apply Restoration Algorithm

  Noise Removal

    Median Filter
    Wiener Filter
    Non-Local Means
    Deep Learning Denoisers

  Deblurring

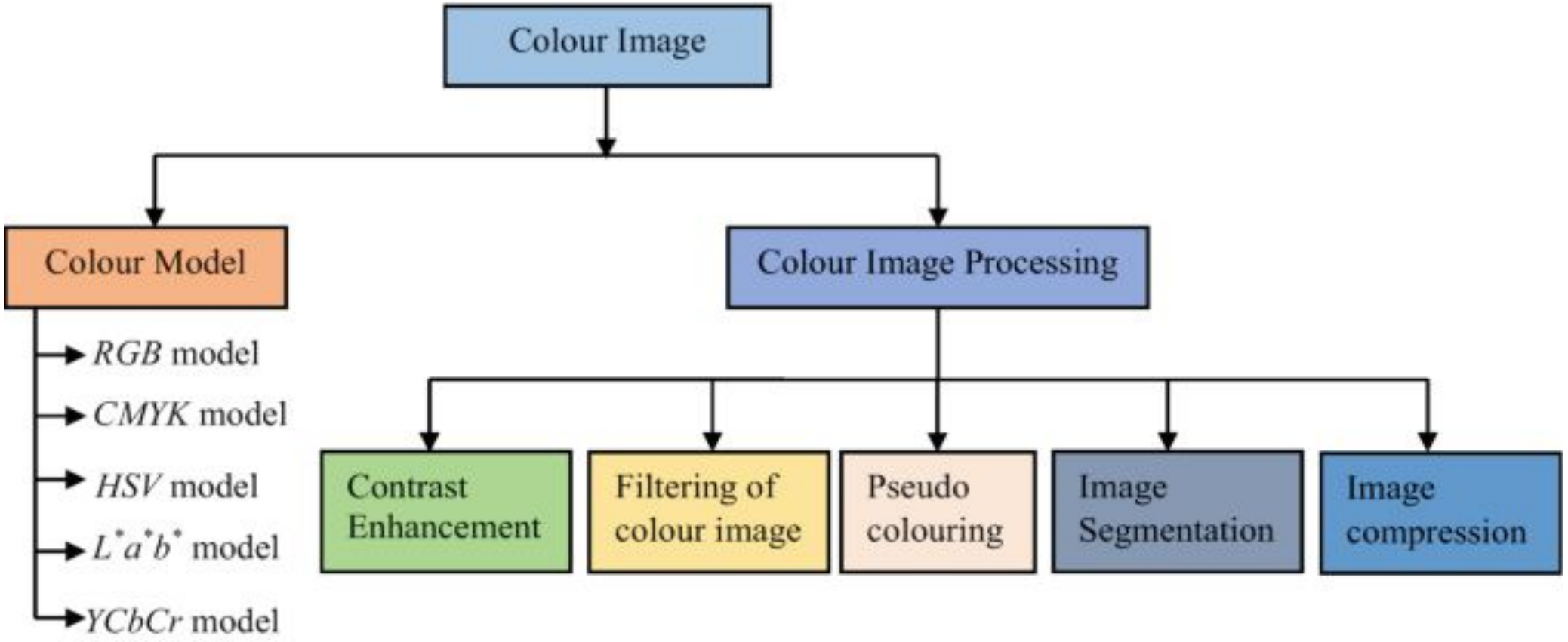
    Inverse Filtering
    Wiener Filtering
    Blind Deconvolution

Inpainting: Fills in damaged or missing regions by estimating surrounding information.

Super-Resolution: Reconstructs a high-resolution image from one or more low-resolution images.

Color image processing

- Process color images with various color models for analysis and correction



- **RGB (Red, Green, Blue):** Used in digital cameras, monitors, and computer vision.
- **CMYK (Cyan, Magenta, Yellow, Black):** Used in color printing.
- **HSV (Hue, Saturation, Value):** Represents colors in a way similar to human perception and is widely used in image segmentation.
- **YCbCr:** Separates brightness from color information and is commonly used in image and video compression.
- **Lab (CIELAB):** Designed to represent colors closer to human visual perception and used in color analysis.

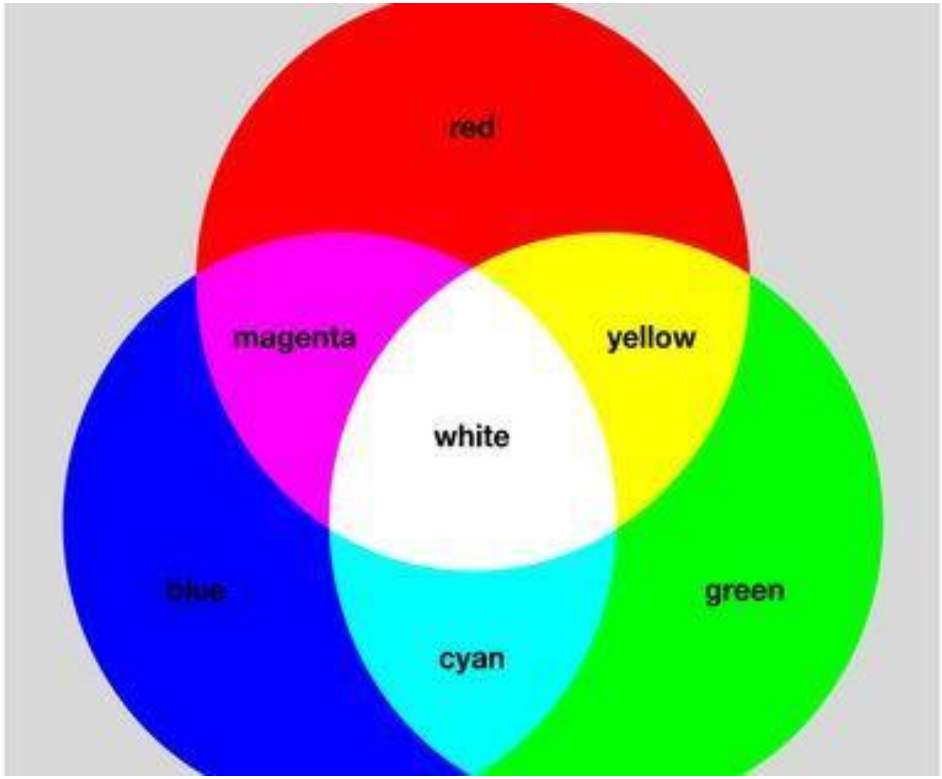


Image compression

- **Image Compression** is the process of reducing the size of an image file by removing redundant or unnecessary information while maintaining acceptable image quality.

Why Do We Need Image Compression?

- Large image files require:
  - More storage space
  - More bandwidth for transmission
  - Longer upload and download times
  - Higher processing cost
- Image compression addresses these challenges by reducing file size.
- Image editing, Text adding
- Types: Lossy and Lossless



Types of Redundancy Removed

- **Spatial Redundancy:** Neighboring pixels often have similar values, so repeated information can be reduced.
- **Coding Redundancy:** Uses more efficient encoding methods to represent pixel data.
- **Psycho-visual Redundancy:** Removes image details that are less noticeable to the human eye.

Types of Image Compression

Lossless Compression

No image information is lost.

The original image can be perfectly reconstructed after decompression.

Examples: PNG: Portable Network Graphics, TIFF: Tagged Image File Format

GIF: Graphics Interchange Format

Applications: Medical imaging, Scientific imaging, Document storage

Lossy Compression

Achieves much higher compression ratios.

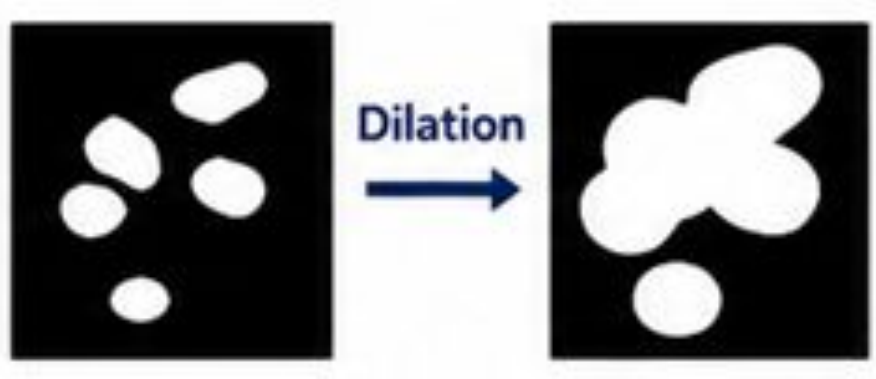
Slight loss of quality, often imperceptible to the human eye.

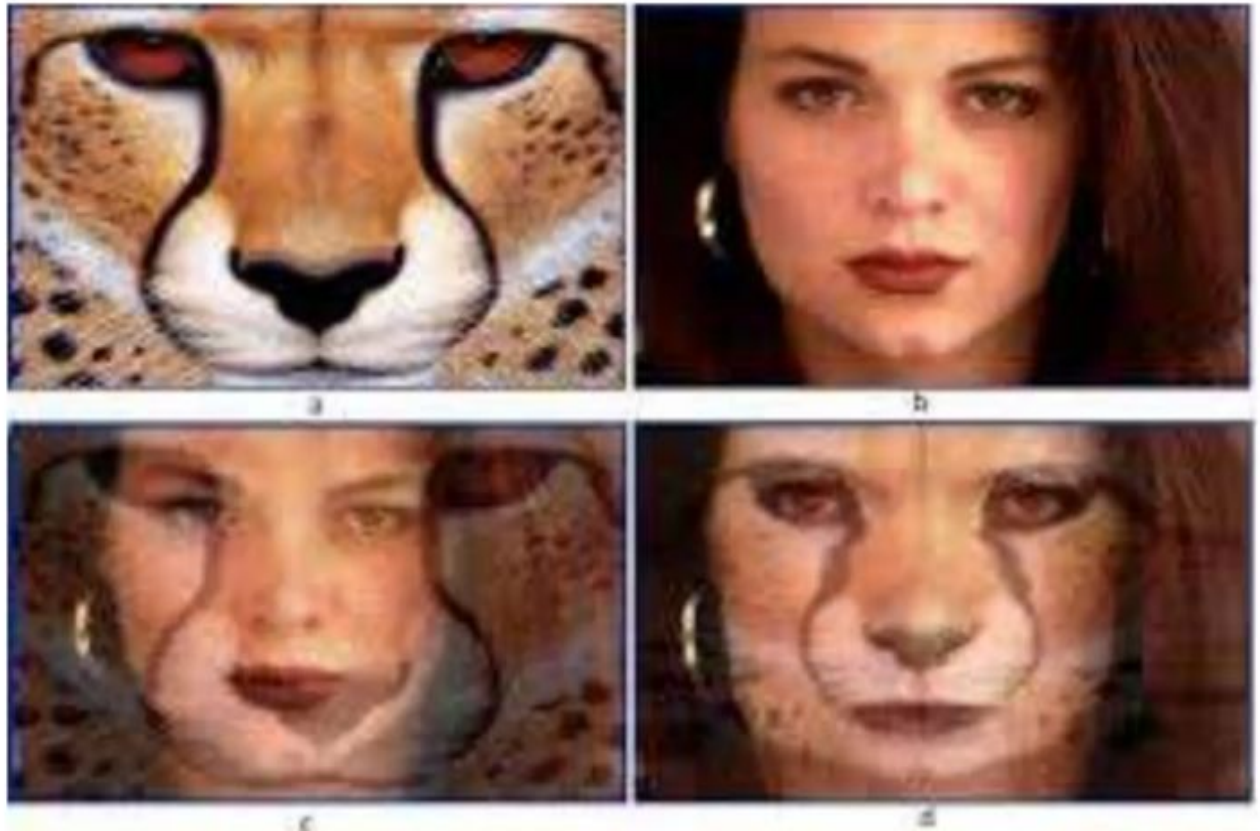
Examples: JPEG

Applications: Digital photography, Websites, Social media, Multimedia

Morphological operations

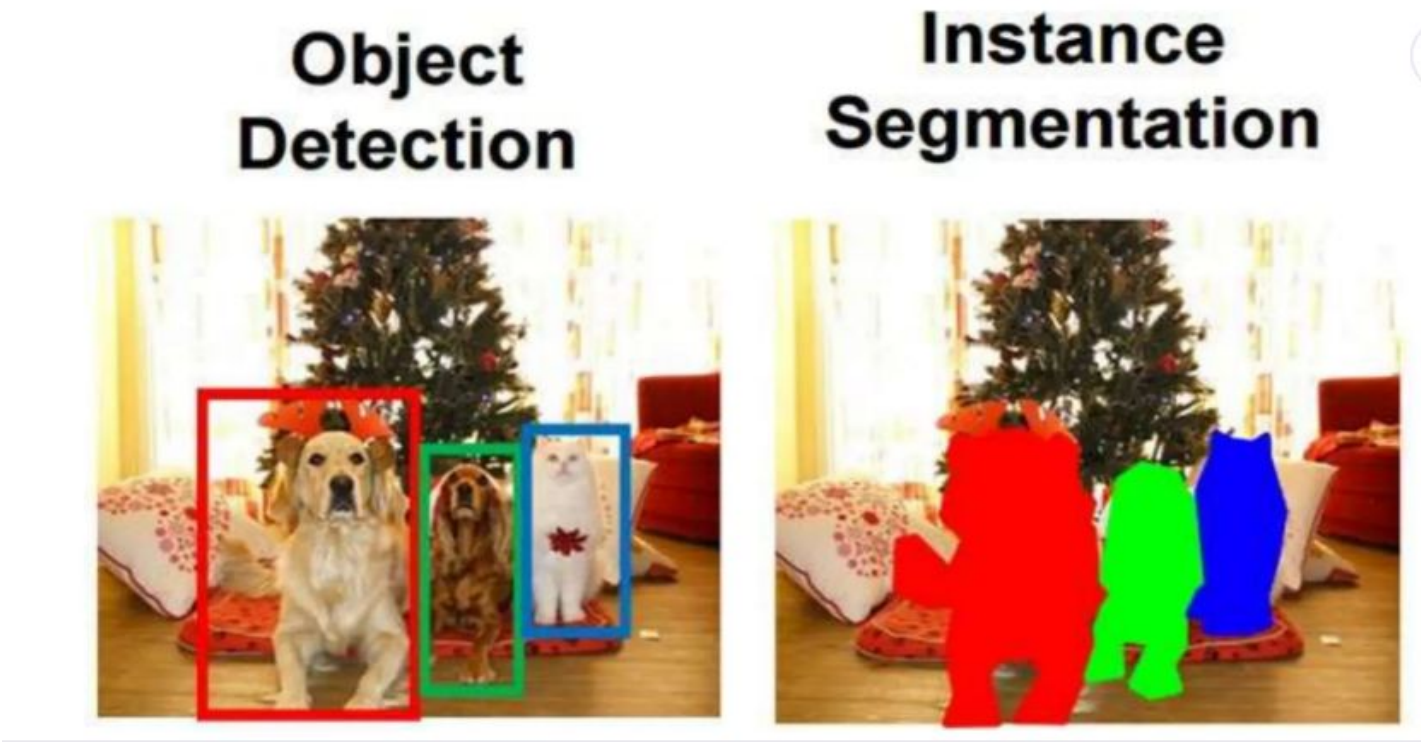
- Morphological operations are image processing techniques that modify the shape, size, and structure of objects in an image using a small predefined pattern called a structuring element (kernel).Unlike filtering operations (which modify pixel intensities), morphological operations focus on the geometrical structure of objects.
- Common operations include:
  - erosion
  - dilation
  - opening
  - closing



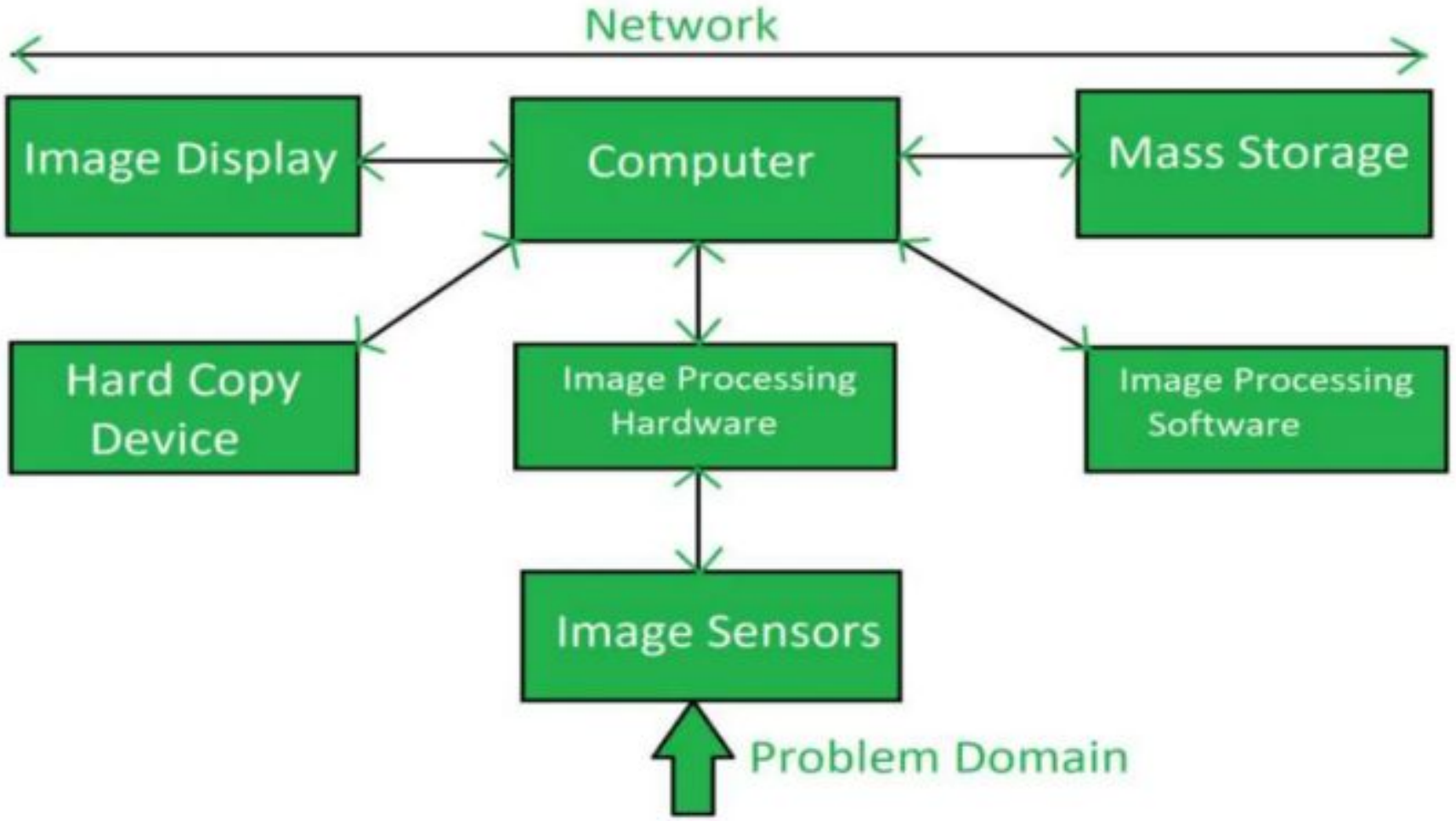


Segmentation

- Partitions an image into meaningful regions corresponding to different objects or structures.
- Methods include:
  - thresholding
  - edge-based segmentation
  - region growing
  - watershed transformation
  - deep learning-based segmentation



Components of Image processing



Basic relationship between pixels

- Each pixel in digital image has a spatial relationship with its neighboring pixels.
- These relationships allow computers to analyze image structures, detect object boundaries, identify connected regions, and extract meaningful information.
- Most image processing algorithms do not analyze pixels individually.
- Instead, they analyze
  - neighboring pixels
  - local intensity variations
  - spatial connectivity

3. 4-NEIGHBORHOOD (N_4)

The 4-neighborhood of a pixel $p(x, y)$ consists of the four pixels that share a common edge with $p(x, y)$.

Pattern

	$(x, y-1)$	
$(x-1, y)$	$p(x, y)$	$(x+1, y)$
	$(x, y+1)$	

Mathematical Representation

$$N_4(p) = \{ (x-1, y), (x+1, y), (x, y-1), (x, y+1) \}$$

Example

5	7	8
9	P	6
2	1	3

→ 4-neighbors of P: {7, 9, 6, 1}

Number of neighbors: 4
(Top, Bottom, Left, Right)

4. DIAGONAL NEIGHBORHOOD (N_D)

The diagonal neighborhood of a pixel $p(x, y)$ consists of the four pixels that touch $p(x, y)$ only at the corners.

Pattern

$(x-1, y-1)$		$(x+1, y-1)$
	$p(x, y)$	
$(x-1, y+1)$		$(x+1, y+1)$

Mathematical Representation

$$N_D(p) = \{ (x-1, y-1), (x+1, y-1), (x-1, y+1), (x+1, y+1) \}$$

Example

1	2	3
4	P	5
6	7	8

→ Diagonal neighbors of P: {1, 3, 6, 8}

Number of neighbors: 4
(Four diagonal pixels)

5. 8-NEIGHBORHOOD (N_8)

The 8-neighborhood of a pixel $p(x, y)$ includes all the eight surrounding pixels (combining N_4 and N_D).

Pattern

$(x-1, y-1)$	$(x, y-1)$	$(x+1, y-1)$
$(x-1, y)$	$p(x, y)$	$(x+1, y)$
$(x-1, y+1)$	$(x, y+1)$	$(x+1, y+1)$

Mathematical Representation

$$N_8(p) = N_4(p) \cup N_D(p)$$
$$= \{ \text{all eight surrounding pixels} \}$$

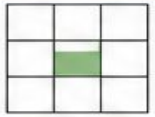
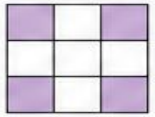
Example

1	2	3
4	P	5
6	7	8

→ 8-neighbors of P: {1, 2, 3, 4, 5, 6, 7, 8}

Number of neighbors: 8
(Top, Bottom, Left, Right and Four Diagonals)

6. COMPARISON OF NEIGHBORHOODS

Neighborhood	Pattern	Pixels Included	Number of Neighbors	Connectivity Used
4-Neighborhood (N_4)		Top, Bottom, Left, Right (Edge neighbors)	4	4-Connectivity
Diagonal Neighborhood (N_D)		Four Diagonal Pixels only	4	(Used in 8- and m-Connectivity)
8-Neighborhood (N_8)		All Surrounding Pixels	8	8-Connectivity

Key Point: Neighborhood defines which pixels are around a given pixel. These relationships are fundamental for adjacency, connectivity, region growing, edge detection, morphological operations, and many other image processing tasks.

Adjacency and Connectivity

- Neighborhood simply tells us which pixels surround a given pixel.
- Adjacency tells us whether two pixels should be considered connected.
- This concept is essential in segmentation and connected component analysis.
- The Two Conditions for Adjacency

For two pixels, $p\ and\ q$, to be adjacent, they must meet both of the following requirements:

- Spatial Proximity: Pixel q must be within a defined neighborhood of pixel p.
- Value Similarity: The intensity values of both p and q must belong to a designated set of gray-level values, denoted as V.

PIXEL ADJACENCY IN DIGITAL IMAGES

Two pixels p and q with the **same value** (or belonging to the same set V) are said to be **adjacent** if one pixel lies in a specified neighborhood of the other.

1. 4-ADJACENCY

Pixels p and q are 4-adjacent if q lies in the **4-neighborhood** of p.

	q	
q	p	q
	q	

4-Neighbors of p

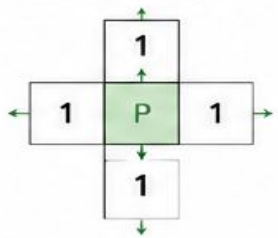
$$N_4(p) = \{(x-1, y), (x+1, y), (x, y-1), (x, y+1)\}$$

Example

Binary Image

0	1	0	0
1	P	0	1
0	1	1	0
0	0	0	1

4-Adjacency



In 4-adjacency, diagonal pixels are NOT considered adjacent.
Path exists only through horizontal or vertical moves.

Used in: Region growing, Connected component labeling (4-connectivity), Morphological operations

2. 8-ADJACENCY

Pixels p and q are 8-adjacent if q lies in the **8-neighborhood** of p.

q	q	q
q	p	q
q	q	q

8-Neighbors of p

$$N_8(p) = N_4(p) \cup N_D(p)$$

(All eight surrounding pixels)

Example

Binary Image

0	1	0	0
1	P	0	1
0	1	1	0
0	0	0	1

8-Adjacency

1	1	1
1	P	1
1	1	1

In 8-adjacency, both horizontal/vertical and diagonal pixels are considered adjacent.
More connections are allowed.

Used in: Segmentation, Boundary tracing, Connected component labeling (8-connectivity)

3. m-ADJACENCY (MIXED ADJACENCY)

Pixels p and q are m-adjacent if

- q is in the 4-neighborhood of p, OR
- q is in the diagonal neighborhood of p AND the pixels that are common 4-neighbors of p and q do NOT belong to set V.

Illustration

q		q
x	p	x

For diagonal adjacency between p and q, the common 4-neighbors (x) must NOT belong to V (i.e., must be 0 in binary images).

Examples (Binary Images)

Case 1: m-adjacent
(common 4-neighbors are 0)

1	0	1
0	P	0
1	0	1

p and q (diagonal) are m-adjacent.

Case 2: Not m-adjacent
(common 4-neighbors are 1)

1	1	1
1	P	1
1	1	1

p and q are NOT m-adjacent.

Used in: Boundary detection, Contour tracing, Thinning, Digital topology applications

Key Takeaway:

- 4-adjacency → only up, down, left, right neighbors
- 8-adjacency → all 8 surrounding neighbors
- m-adjacency → avoids ambiguity in diagonal connections

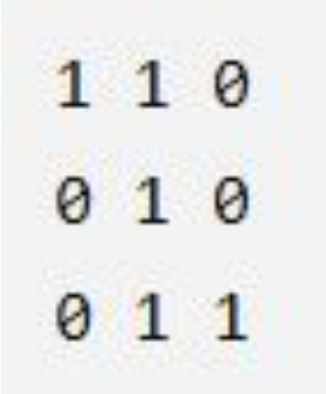
Path

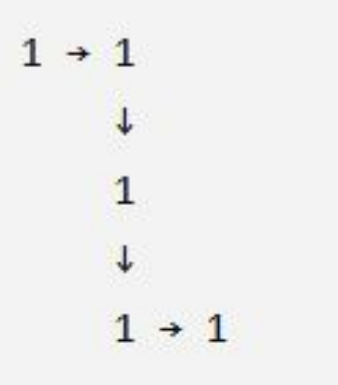
- A **digital path** is a sequence of connected pixels joining one pixel to another.
- If $p = (x0, y0)\ and\ q = (xn, yn)$ then a path is

$$(x0, y0), (x1, y1) , \dots , (xn, yn)$$

where every consecutive pair of pixels is adjacent.

- The number of links is called the **path length**.
- If the starting and ending pixels are identical, it is called a **closed path**.





Connectivity

- Let S represent a subset of pixels in an image, two pixels p and q are said to be connected in S if there exists a path between them consisting entirely of pixels in S.
- For any pixel p in S, the set of pixels that are connected to it in S is called a connected component of S. If it only has one connected component, then set S is called a connected set.

Distance Measure

- A **distance measure** is a mathematical way of calculating the distance between two points in an image.
- For pixels p, q, and s, with coordinates (x, y) , (u, v)and (w, z)respectively, D is a distance function or metric if

$$\begin{aligned}
&\textbf{(a)}\ D(p,q) \geq 0 \quad (D(p,q) = 0 \ \text{iff} \ \ p = q), \\
&\textbf{(b)}\ D(p,q) = D(q,p), \text{ and} \\
&\textbf{(c)}\ D(p,s) \leq D(p,q) + D(q,s).
\end{aligned}$$

- Distance measure are used in various Segmentation, Morphological, Edge detection, object tracking, pattern recognition task.

Types of distances

- Euclidean distance: This is the shortest (straight-line) distance between two points.

$$D_E = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- City Block distance (Manhattan Distance)

- Chessboard Distance

$$D_C = \max(|x_2 - x_1|, |y_2 - y_1|)$$

Example

- Let A = (1,1) and B = (5,4)
- Euclidean Distance :

$$D_E = \sqrt{(5-1)^2 + (4-1)^2}$$
$$= \sqrt{4^2 + 3^2} = 5$$

- City Block

$$D_M = |5-1| + |4-1| \quad = 7$$

- Chessboard

$$D_C = \max(4, 3) = 4$$

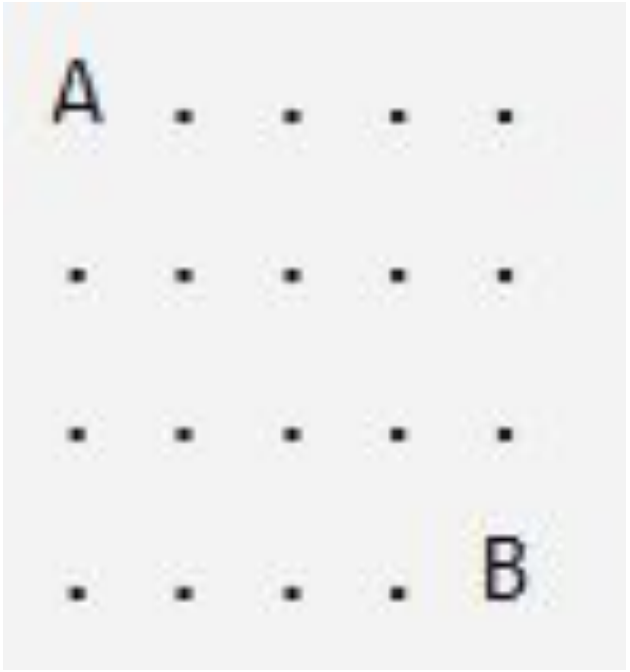


Image Interpolation

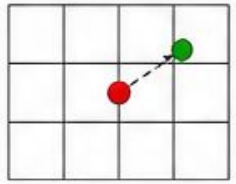
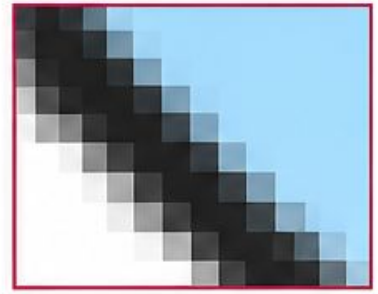
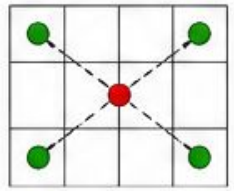
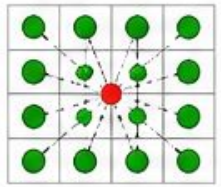
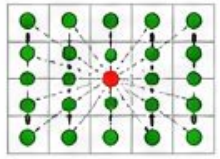
- Interpolation means estimating unknown pixel values using the known values of nearby pixels.
- It is used to estimate new pixel values when an image is enlarged (zoomed), reduced (shrunk), rotated, or geometrically transformed.
- **Example:**

  Original image size = **500 × 500 pixels**

  Enlarged image size = **750 × 750 pixels**

Since the larger image needs more pixels than the original, interpolation calculates the values of these new pixels based on the existing ones.

Interpolation Methods and Their Visual Effects

Method	How it Works	Example (Zoomed Image)	Close-up (Edge)	Visual Effect
1. Nearest Neighbor Interpolation	Uses the value of the nearest pixel.  New pixel takes the value of nearest neighbor			• Blocky appearance • Jagged edges • Pixelated look • Fastest but lowest quality 😟
2. Bilinear Interpolation	Uses the weighted average of the 4 nearest pixels.  New pixel is weighted average of 4 neighbors			• Smoother than nearest neighbor • Edges look smoother • Slightly blurry • Good balance of speed and quality 😐
3. Bicubic Interpolation	Uses the weighted average of the 16 nearest pixels.  New pixel is weighted average of 16 neighbors			• Smoother and sharper • Better edge preservation • Less blurring • Higher computational cost 🙂
4. Lanczos Interpolation	Uses a sinc function over a larger neighborhood (typically 36+ pixels).  New pixel is computed using sinc function with many neighbors			• Very sharp • Excellent detail preservation • Minimal blurring • Slight ringing (halo) may appear near edges • Slowest but highest quality 😀

Basic mathematical operations on image

- Elementwise Operations (Pixel-by-Pixel)
  - Each pixel is processed independently from the others.
  - Addition, Subtraction, Multiplication, Division

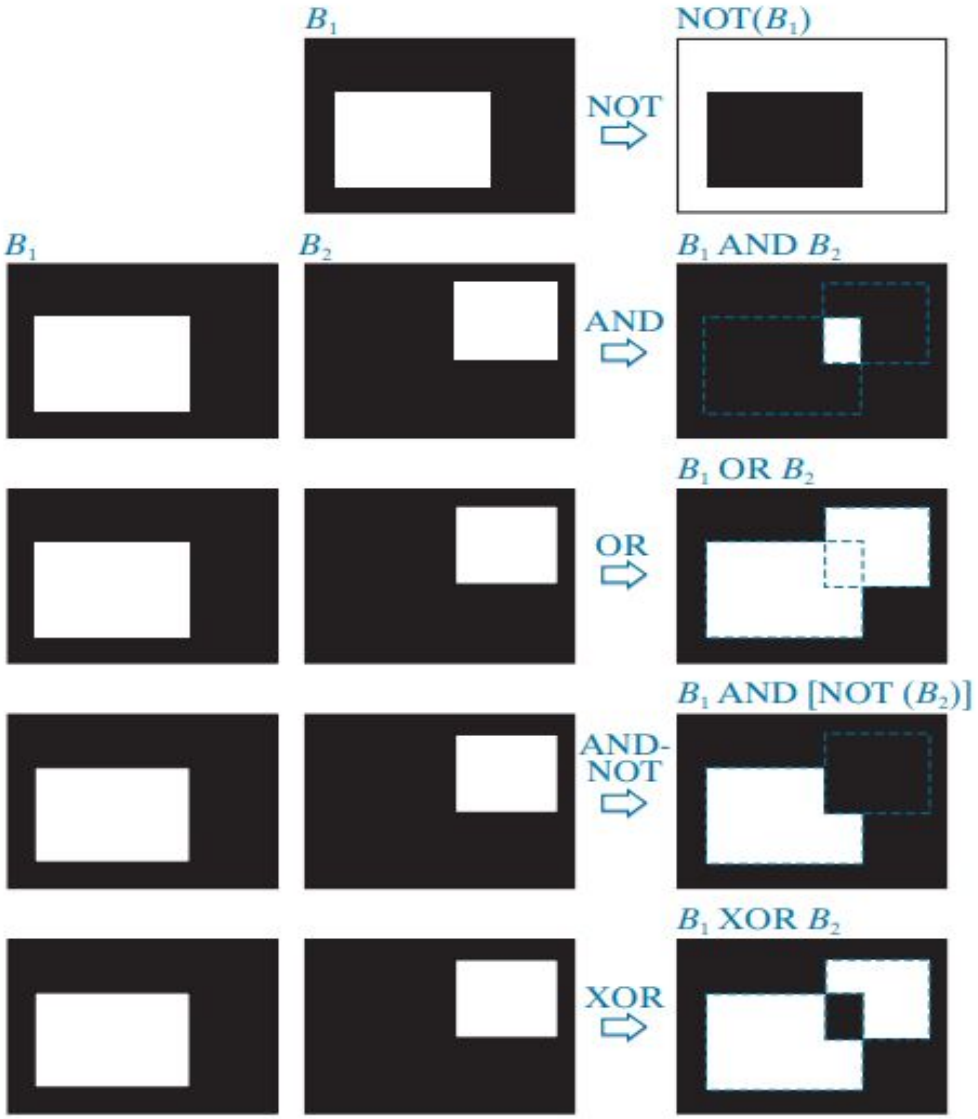
Advantages
- Simple and fast.
- Easy to implement.
- Good for brightness and contrast changes.

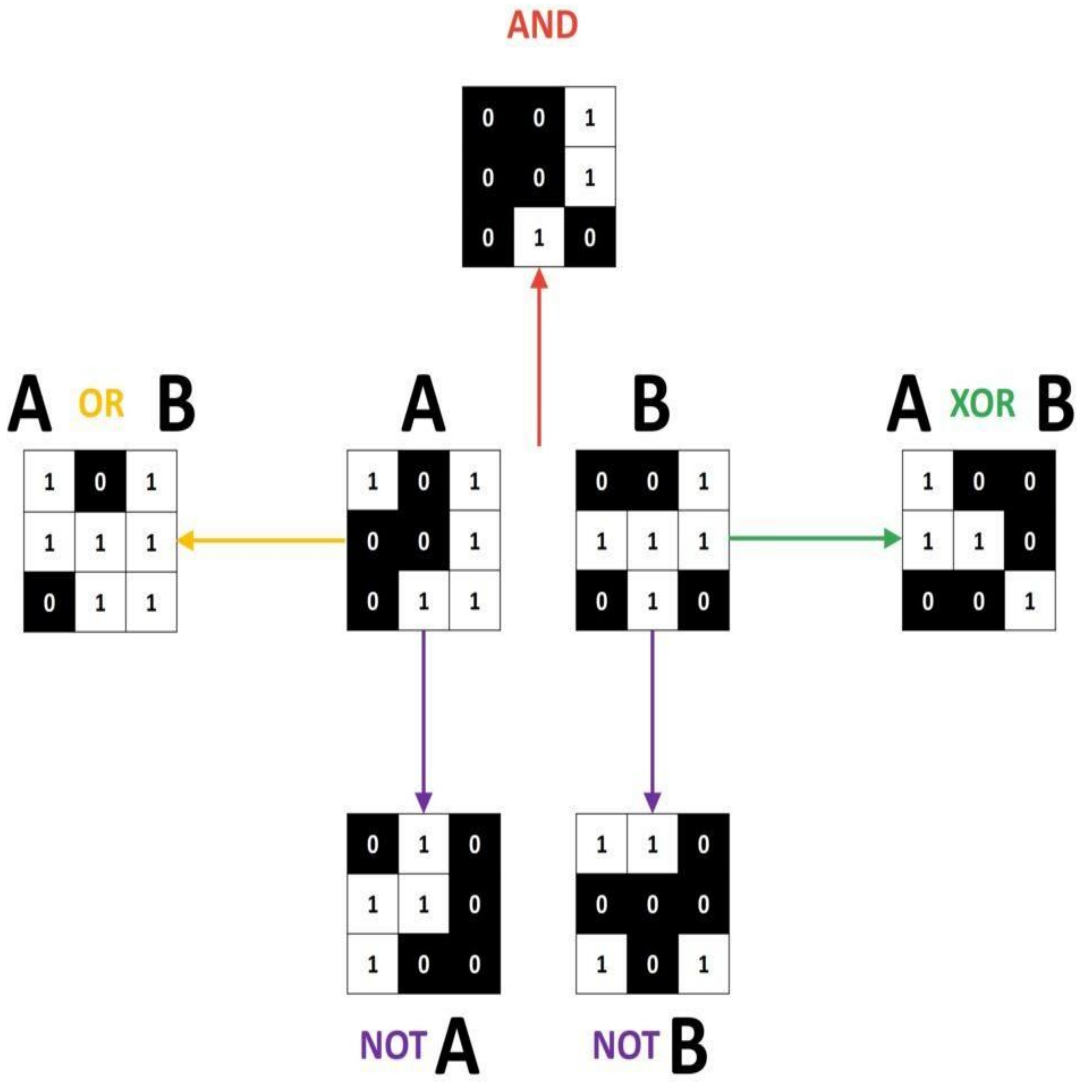
Disadvantages
- Ignores neighboring pixels.
- Cannot remove noise effectively.
- Cannot detect edges or shapes well.

Linear vs Nonlinear Operations

- Linear Operations
  - The output changes proportionally with the input.
  - Addition, Weighted, average, Linear filtering
- Nonlinear Operations
  - The output does not change proportionally with the input.
  - Maximum (Max), Minimum (Min), threshold, median

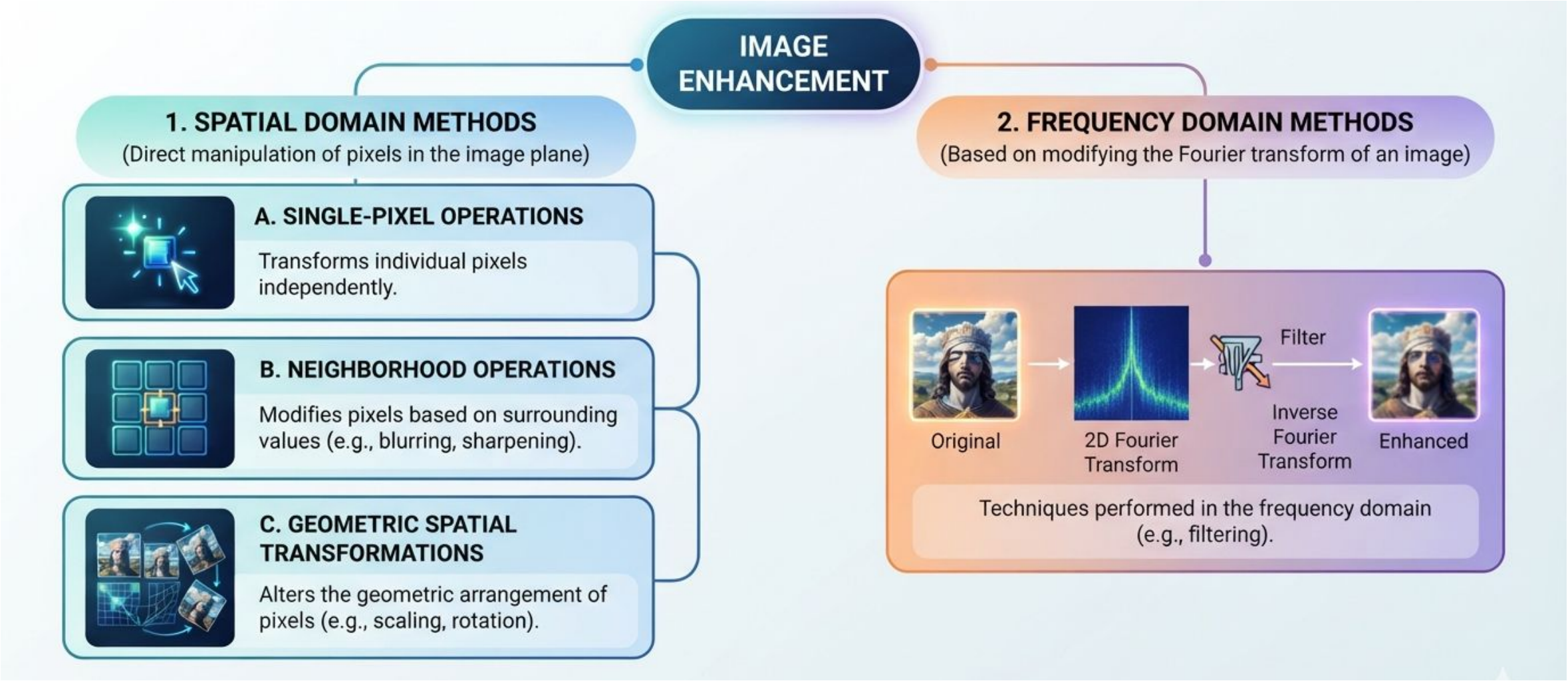
Logical operations





Spatial Domain Transformation

- Enhancing an image provides better contrast and a more detailed image as compare to non enhanced image



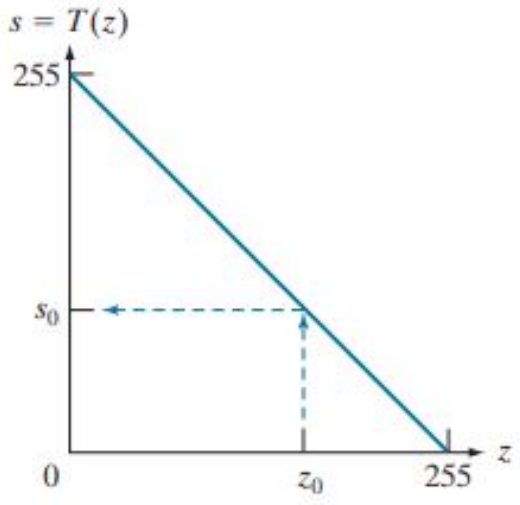
Single-pixel transformation:

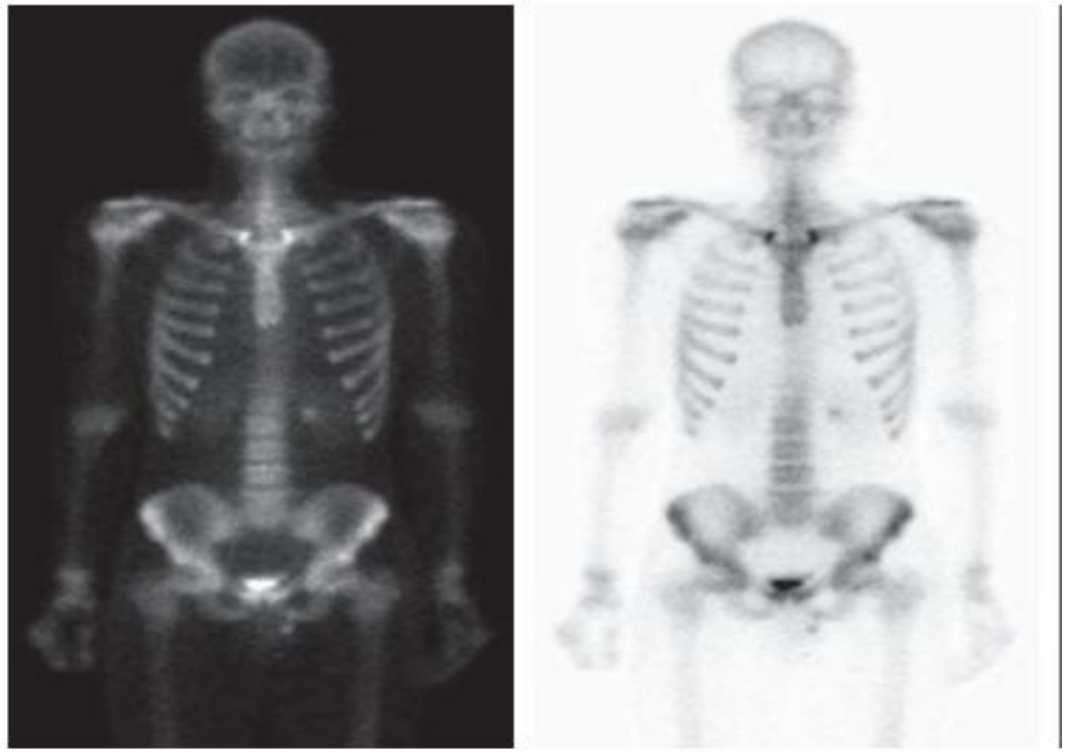
- Modify digital image intensities individually using a transformation function,

$$s = T(z),$$

where z is the original intensity and s is the mapped intensity.

- Example: generating the negative





Neighborhood operation

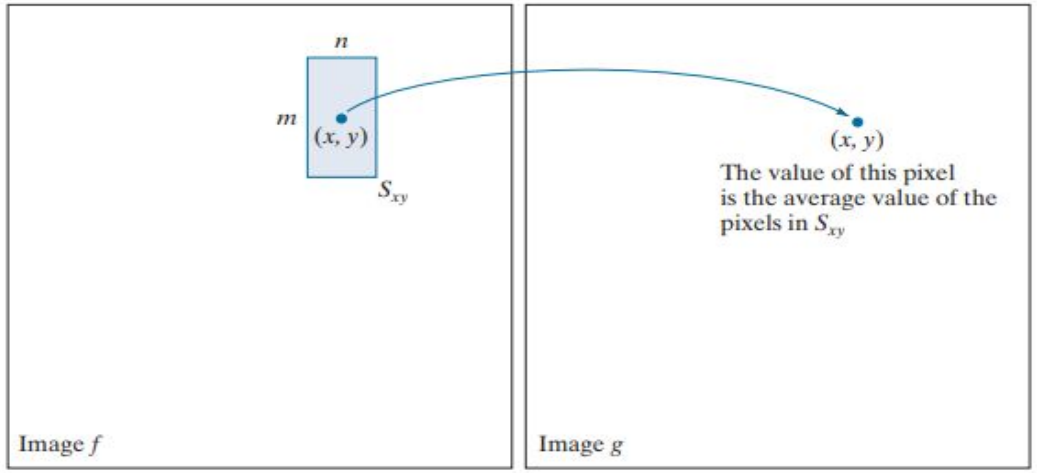
- Neighborhood processing calculates a new pixel value in an output image based on a specified operation applied to a defined neighborhood set.
- Let Sxy denote the set of coordinates of a neighborhood centered on an arbitrary point (x, y) in an image, f.
- suppose that the specified operation is to compute the average value of the pixels in a rectangular neighborhood of size $m \; x \; n$ centered on (x, y).

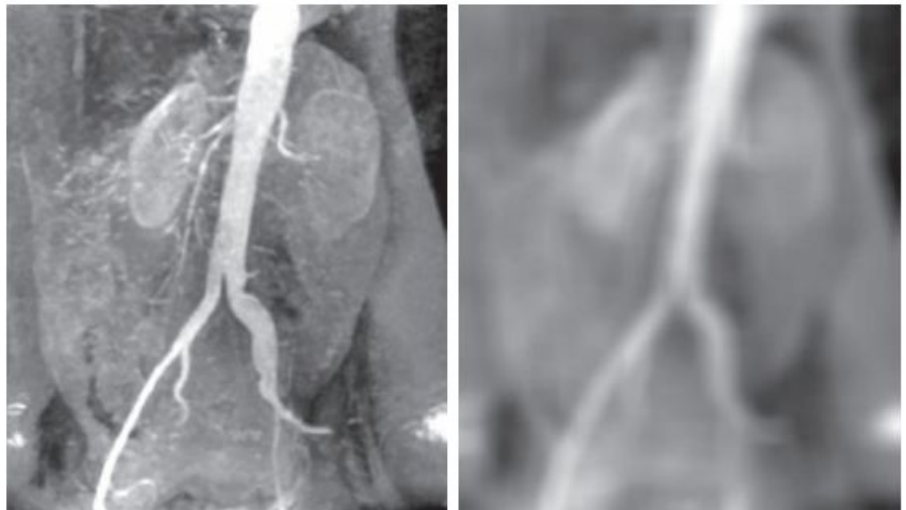
The averaging expression is given as

$$g(x,y) = \frac{1}{mn} \sum_{(r,c) \in S_{xy}} f(r,c)$$

Where, m x n are neighborhood pixels and r, c are

Rows and columns coordinates of the pixels whose coordinates are in the set Sxy





Translation Transformation

- **Translation** is the process of shifting every pixel in an image by a fixed distance along the horizontal and/or vertical directions without changing its orientation, size, or shape.
- Translation simply changes the **position** of the object. It do not change shape, size, distance and orientation of the image.

Mathematical Representation

Suppose a pixel is located at $P(x, y)$ and we wish to move it by t_x units in the x-direction and t_y units in the y-direction.

The new coordinates become

$$x' = x + t_x$$
$$y' = y + t_y$$

where

t_x = horizontal translation

t_y = vertical translation

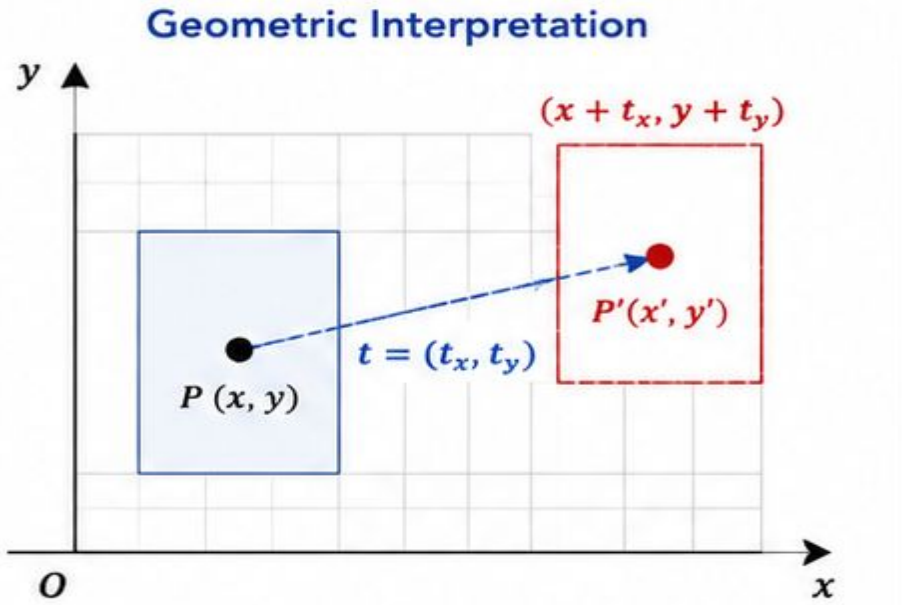
Vector Form

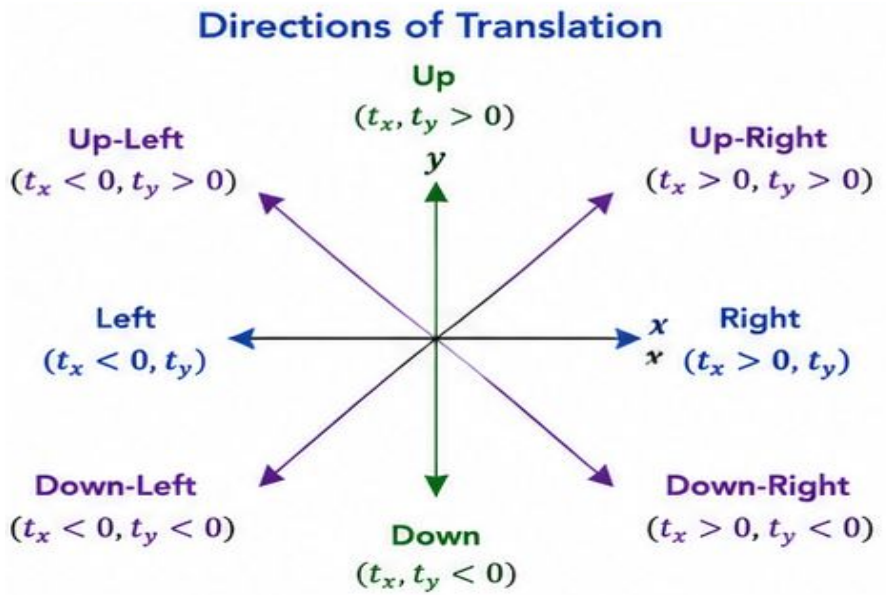
Translation can also be written as

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

The translation vector $t = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$

determines the direction and magnitude of the shift.





- Real-Life Examples
  - Moving an image on a PowerPoint slide.
  - Shifting a satellite image to align it with another image.
  - Repositioning an object in image editing software.
  - Aligning multiple frames in video processing.
- Example:

A pixel is located at P(4,5) Translate it by tx =3, ty =2

$P(4,5) \rightarrow P'(7,7)$

Scaling Transformation

- **Scaling** is a geometric transformation that changes the size of an object by multiplying its coordinates by scaling factors along the x-axis and y-axis.
  - If the scaling factors are greater than 1, the object becomes larger.
  - If the scaling factors are less than 1, the object becomes smaller.
- If point P (x, y) is scaled by Sx (along x axis) and Sy (along y-axis) the new coordinate becomes

$$x' = S_x \; x$$
$$y' = S_y \; y$$

Matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Vector form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} - \begin{bmatrix} S_x \; x \\ S_y \; y \end{bmatrix}$$

SCALING OF A POINT: EXAMPLE

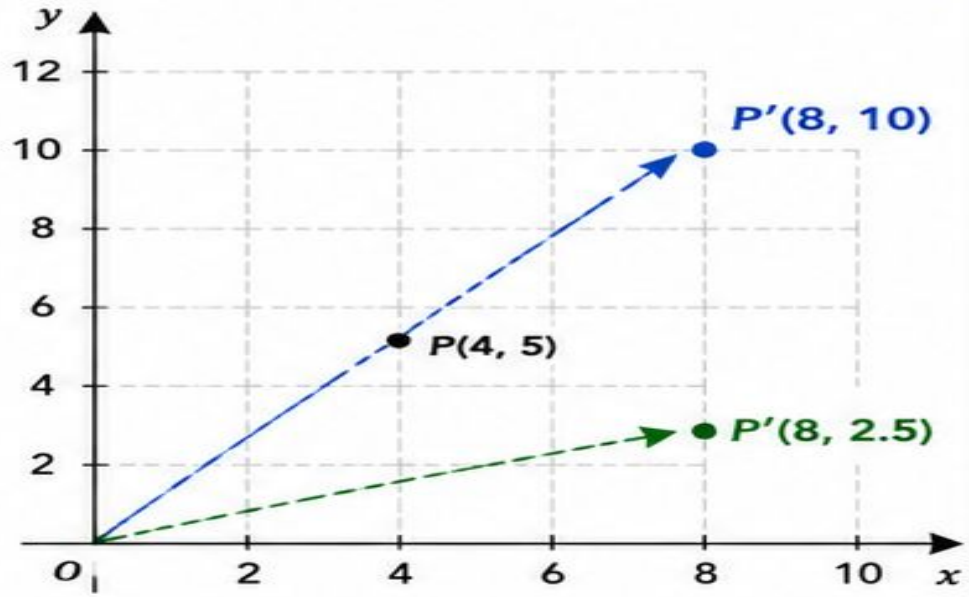
Given point $P(4, 5)$

Example 1: Uniform Scaling ($S_x = S_y = 2$)

$$\left.\begin{aligned} x' &= 2 \times 4 = 8 \\ y' &= 2 \times 5 = 10 \end{aligned}\right\} \Rightarrow P'(8, 10)$$

Example 2: Non-uniform Scaling ($S_x = 2, S_y = 0.5$)

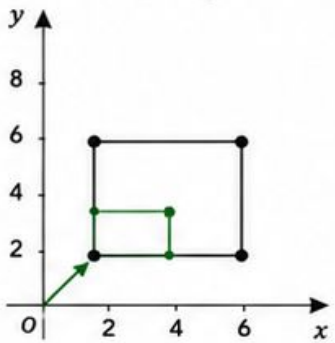
$$\left.\begin{aligned} x' &= 2 \times 4 = 8 \\ y' &= 0.5 \times 5 = 2.5 \end{aligned}\right\} \Rightarrow P'(8, 2.5)$$



VISUAL EFFECT OF SCALING ABOUT ORIGIN (0, 0)

Reduction (0 < S < 1)

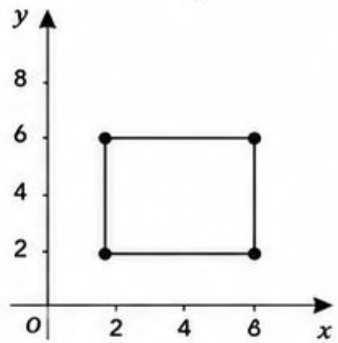
$S_x = S_y = 0.5$



Object becomes smaller

No Change (S = 1)

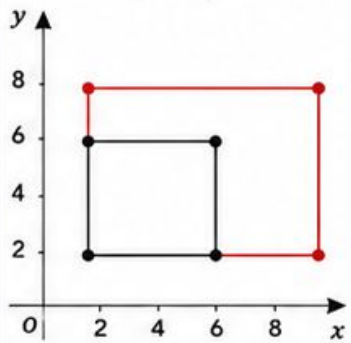
$S_x = S_y = 1$



Object remains the same

Enlargement (S > 1)

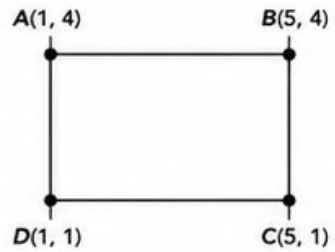
$S_x = S_y = 2$



Object becomes larger

EXAMPLE: VISUAL CHANGE IN SHAPE (Non-uniform Scaling)

Original Object (Rectangle)



Width = 4 units

Height = 3 units

Scaling Factors:

$S_x = 2$, $S_y = 0.5$

After Scaling (Shape Changed)



Width = 8 units

Height = 1.5 units

• Scaling About an Arbitrary Point

- In many practical applications, we want to scale an object about its center rather than about the origin.
- This is achieved using three steps:
    1. Translate the object so that the center coincides with the origin.
    2. Apply the scaling transformation.
    3. Translate the object back to its original position.

$$T^{-1}\,S\,T$$

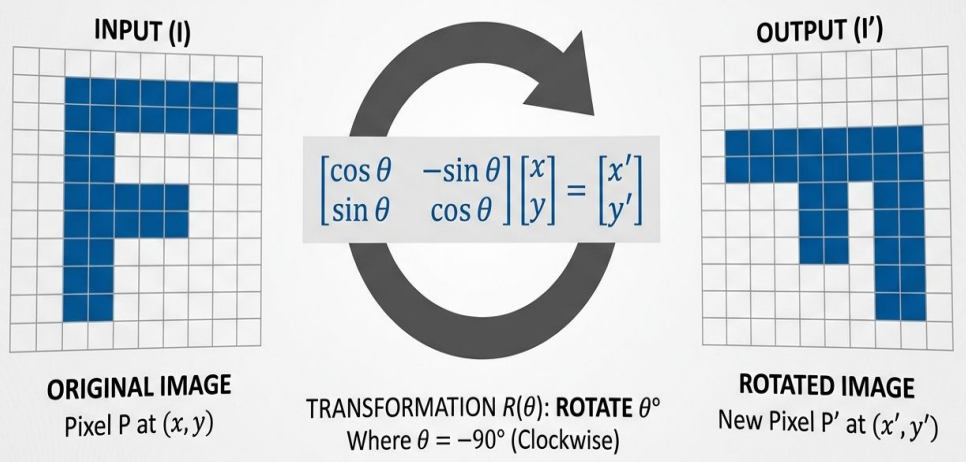
- This concept is extensively used in image editing software.

Real-Life Example

- Zooming a Photograph : When you zoom into a photograph on your smartphone, the displayed image becomes larger. This is achieved through scaling.
- Google Maps : When zooming in or out on Google Maps, the map is scaled without changing the geographical information.
- Medical Imaging: Doctors enlarge MRI or CT scan images to observe fine anatomical details.

Rotational Transformation

- A **Rotation Transformation** is a geometric transformation that rotates every point of an object through a specified angle about a fixed point called the center of rotation.
- Usually, unless otherwise specified, the center of rotation is the **origin (0,0)**.



5. MATHEMATICAL REPRESENTATION

If a point $P(x, y)$ is rotated by an angle θ about the origin $(0, 0)$, the new coordinates $P'(x', y')$ are

$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$

where θ = angle of rotation

Counterclockwise rotation	→ θ is positive $(+\theta)$
Clockwise rotation	→ θ is negative $(-\theta)$

6. MATRIX REPRESENTATION

Rotation can be written in matrix form as

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

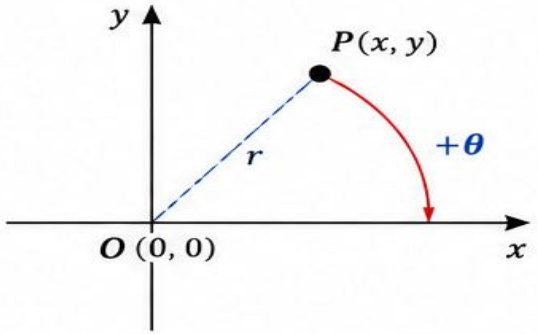
Here, the rotation matrix is

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

7. DIRECTION OF ROTATION

(A) Counterclockwise (Positive Rotation)

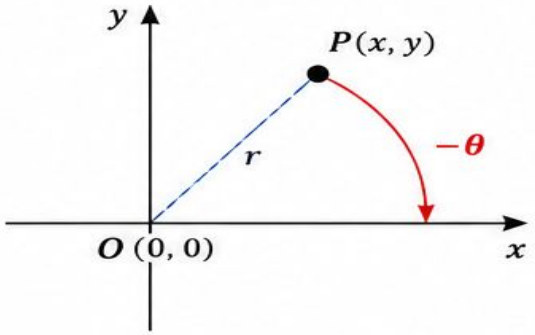
$\theta > 0$



Rotation is in counterclockwise direction and angle θ is positive $(+\theta)$.

(B) Clockwise (Negative Rotation)

$\theta < 0$



Rotation is in clockwise direction and angle θ is negative $(-\theta)$.

Standard rotation matrices:

$\theta = 90°$ (counterclockwise)	$\theta = 180°$	$\theta = 270°$ (counterclockwise)
$R(90°) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$	$R(180°) = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$	$R(270°) = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$

Real-Life Examples

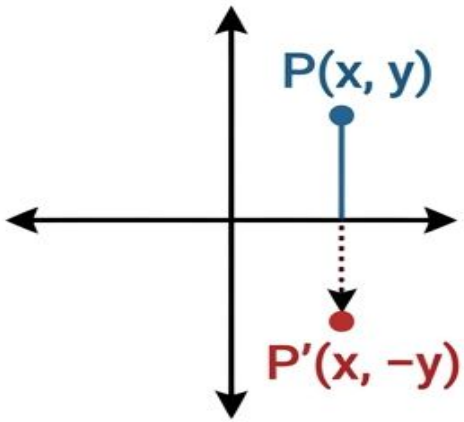
- **Mobile Phone:** When you rotate your smartphone from portrait mode to landscape mode, the displayed image undergoes a rotation transformation.
- **Google Maps:** The navigation map rotates according to the direction in which you are driving.
- **Medical Imaging:** Doctors rotate CT or MRI scans to observe anatomical structures from different orientations.
- **Robotics:** A robot rotates camera images before recognizing objects.
- OCR (Optical Character Recognition): Scanned documents that are slightly tilted are rotated to make the text horizontal before character recognition.

Reflection Transformation

- A **Reflection Transformation** is a geometric transformation in which every point of an object is mapped to the opposite side of a specified line (called the **axis of reflection**) while maintaining the same perpendicular distance from that line.
- The reflecting line acts like a mirror.
- **Real-Life Examples**
  - Selfie cameras that display mirrored previews.
  - Horizontal or vertical image flipping in Photoshop.
  - Data augmentation for machine learning.
  - Medical image visualization.
  - Character recognition.

Types of reflections

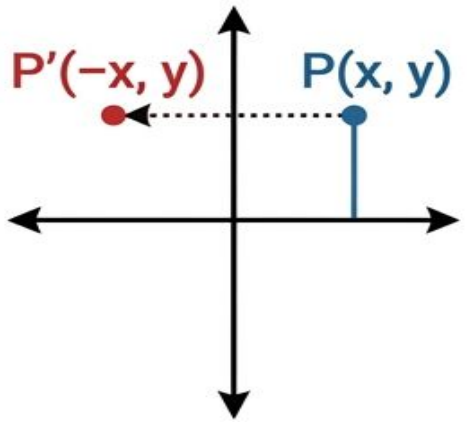
Reflection about X-axis



Equation: $x' = x$
$y' = -y$

Matrix: $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

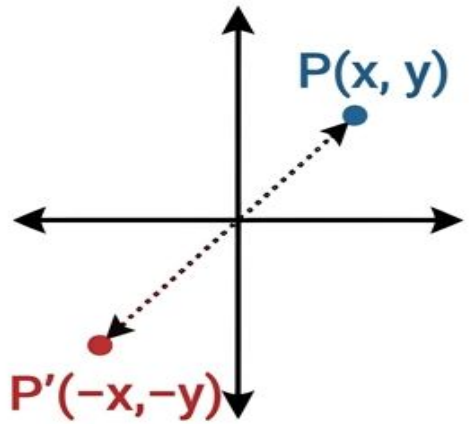
Reflection about Y-axis



Equation: $x' = -x$
$y' = y$

Matrix: $\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$

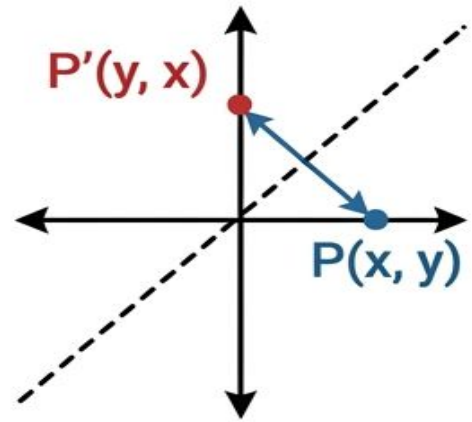
Reflection about Origin



Equation: $x' = -x$
$y' = -y$

Matrix: $\begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix}$

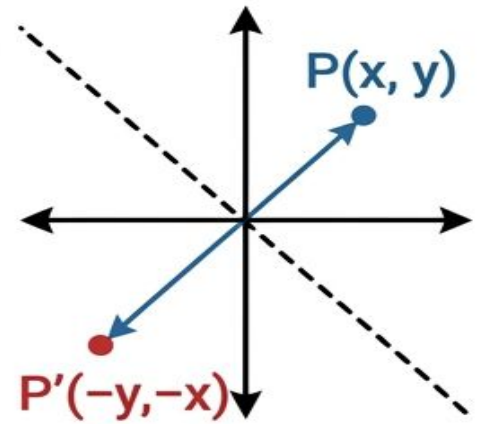
Reflection about Line y=x



Equation: $x' = y$
$y' = x$

Matrix: $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

Reflection about Line y=−x

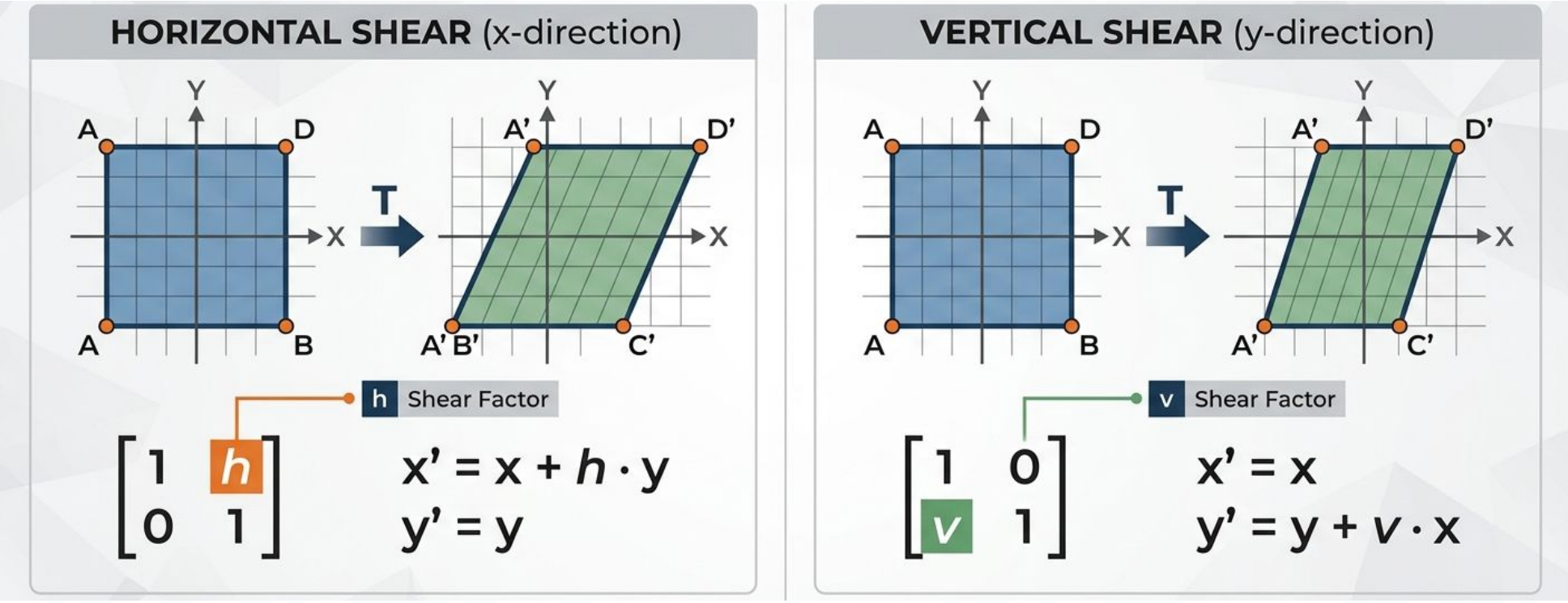


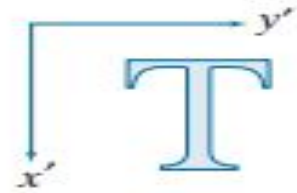
Equation: $x' = -y$
$y' = -x$

Matrix: $\begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$

Shearing (Skewing) Transformation

- A **Shearing Transformation** is a geometric transformation in which one coordinate of every point is displaced proportionally to the other coordinate, causing the object to be slanted while preserving parallelism.



Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = y$	
Scaling/Reflection (For reflection, set one scaling factor to −1 and the other to 0)	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = c_x x$ $y' = c_y y$	
Rotation (about the origin)	$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x\cos\theta - y\sin\theta$ $y' = x\sin\theta + y\cos\theta$	
Translation	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + t_x$ $y' = y + t_y$	
Shear (vertical)	$\begin{bmatrix} 1 & s_v & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + s_v y$ $y' = y$	
Shear (horizontal)	$\begin{bmatrix} 1 & 0 & 0 \\ s_h & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = s_h x + y$	

Comparison

Property	Translation	Scaling	Rotation	Reflection	Shearing
Changes Position	✓	✗	✓	✓	✓
Changes Size	✗	✓	✗	✗	✗
Changes Orientation	✗	✓ (non-uniform may distort)	✓	✓ (reversed)	✓ (slanted)
Preserves Distance	✓	✗	✓	✓	✗
Preserves Angles	✓	Uniform only	✓	✓	✗
Preserves Shape	✓	Uniform only	✓	✓	✗
Preserves Parallel Lines	✓	✓	✓	✓	✓

Affine transformation

- Affine transformation combines multiple transformations such as translation, rotation, scaling, and shearing into a single operation. It preserves straight lines and parallel lines, making it one of the most widely used geometric transformations.

- Applications: Image registration, object tracking, and image warping.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = A \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

where:

(x, y) is the input coordinate

(x', y') is the output coordinate

A is the affine transformation matrix

Orthogonal Transformation

- An orthogonal transformation may rotate or reflect an image, but it does not stretch, shrink, or shear it.
- It preserves the original **lengths, angles, distances, and shape** of an object.
- It is represented by an orthogonal matrix QQQ that satisfies:

$$Q^T Q = I$$

where:

- Q^T is the transpose of Q
- I is the identity matrix
- The inverse of an orthogonal matrix is equal to its transpose:

$$Q^{-1} = Q^T$$

Degrees of Freedom (DOF)

- The Degrees of Freedom (DOF) of a transformation indicate the number of independent parameters required to uniquely define that transformation.
- A transformation with more degrees of freedom is more flexible but also more complex.
- Examples
  - Translation: Two parameters (tx, ty) $\longrightarrow$ DOF = 2
  - Rotation: One parameter (θ) $\longrightarrow$ DOF = 1
  - Scaling: Two parameters (Sx, Sy) $\longrightarrow$ DOF = 2
  - Reflection: No continuously variable parameter for a fixed reflection axis (the reflecting axis is chosen from predefined lines).
  - Shearing: Two parameters (Shx,Shy) $\longrightarrow$ DOF = 2

Other Transformation

Transformation Model	Parameters	DOF
Orthogonal	Rotation (or reflection)	1 (rotation only)
Euclidean	Rotation + Translation	3
Similarity	Rotation + Translation + Uniform Scaling	4
Affine	Rotation + Translation + Non-uniform Scaling + Shearing	6
Projective	General 3×3 homography (up to scale)	8

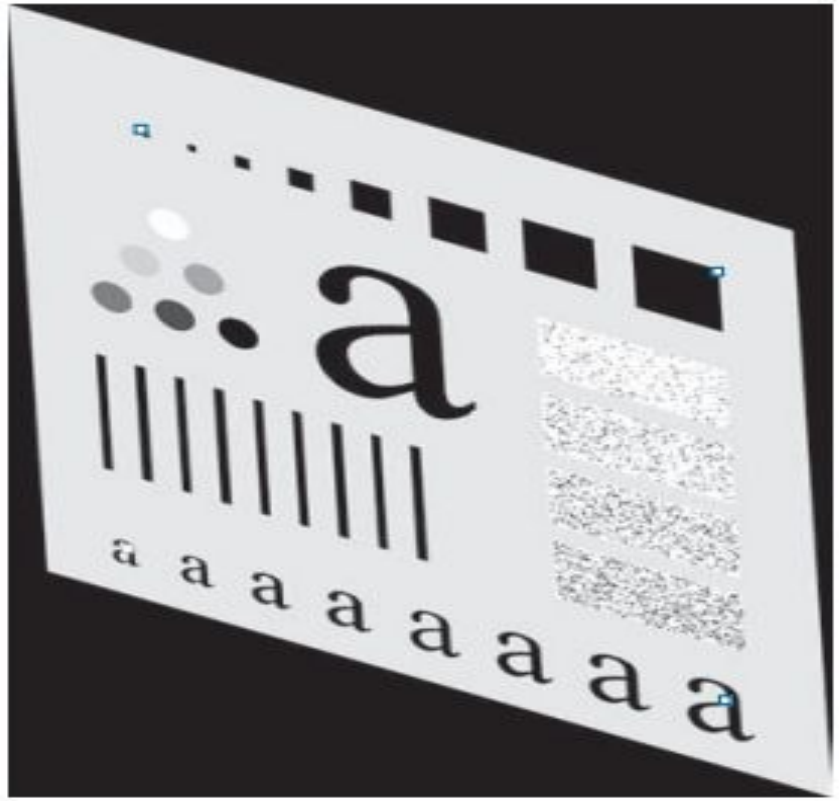
Image registration

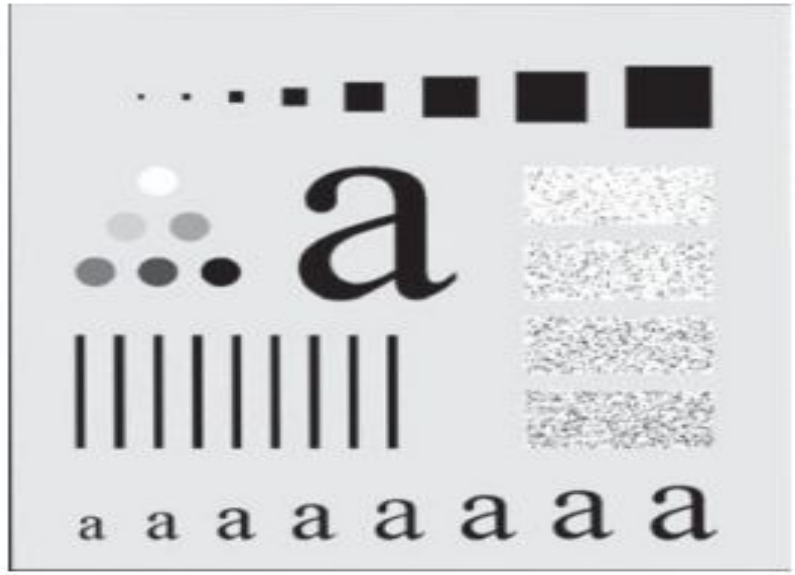
- Image registration is an important application of digital image processing used to align two or more images of the same scene.
- In image registration, we have available an input image and a reference image. The objective is to transform the input image geometrically to produce an output image that is aligned (registered) with the reference image.
- Example: Medical Scans (Cancer Tracking)

a b
c d

FIGURE 2.42
Image registration. (a) Reference image. (b) Input (geometrically distorted image). Corresponding tie points are shown as small white squares near the corners. (c) Registered (output) image (note the errors in the border). (d) Difference between (a) and (c), showing more registration errors.









Spatial Domain Processing

- Spatial Domain Processing refers to image processing techniques in which operations are performed directly on the pixel values (intensities) of an image.
- The general expression is

$$g(x, y) = T[f(x, y)]$$

$f(x, y) =$ Input image

$g(x, y) =$ Output (processed) image

$T =$ Transformation function applied directly on pixel intensities

Example

- Let we have an image of $3\ x\ 3$ pixel size

$$\begin{matrix} 20 & 40 & 55 \\ 60 & 10 & 17 \\ 22 & 90 & 150 \end{matrix}$$

- And we want to increase the brightness, let say by 50

$$\begin{matrix} 70 & 90 & 105 \\ 110 & 80 & 67 \\ 72 & 140 & 200 \end{matrix}$$

Intensity Transformation

- An Intensity Transformation (also called a Gray-Level Transformation) is a spatial domain operation that modifies the intensity value of every pixel according to a predefined mathematical function.
- **Types of intensity transformation**

  Linear

  Nonlinear

Linear Intensity Transformations

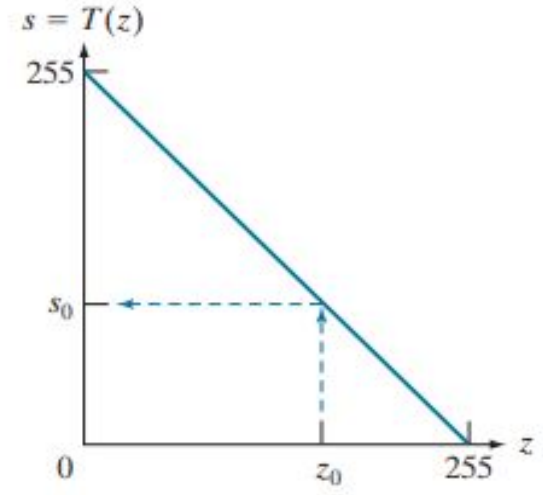
- In linear transformations, the relationship between input and output intensity values follows a straight-line equation. The amount of change remains proportional throughout the intensity range.
- Examples include:
  - Brightness adjustment
  - Contrast stretching
  - Linear scaling
- These techniques are generally used when a uniform modification of image intensity is required.

Nonlinear Transformations

- In nonlinear transformations, the relationship between input and output intensity values is not linear. The transformation curve may bend upward or downward depending on the desired enhancement.
- Examples include:
  - Log transformation
  - Inverse log transformation
  - Gamma transformation
- Nonlinear transformations are particularly useful for emphasizing either dark regions or bright regions of an image.

Image Negative

- Image Negative Transformation is an intensity transformation in which every pixel intensity is replaced by its complementary gray level with respect to the maximum intensity value.



- The transformation changes

  Black (0) → White (255)

  White (255) → Black (0)

- Idea is Dark becomes bright and Bright becomes dark.
- For an L-level image

$$s = (L - 1) - r$$

Algorithm

Step 1: Read the input image.

Step 2: Read the intensity value rrr of each pixel.

Step 3: Apply the transformation

$$s = (L - 1) - r$$

Step 4: Replace the original pixel value with the transformed value.

Step 5: Repeat for all pixels.

Step 6: Display the negative image.

Example

- For Image

$$I = \begin{matrix} 20 & 40 & 55 \\ 60 & 10 & 17 \\ 22 & 90 & 150 \end{matrix}$$

- Image negative

$$I' = \begin{matrix} 235 & 215 & 200 \\ 95 & 245 & 238 \\ 233 & 226 & 105 \end{matrix}$$

Log transformation

- Log Transformation is an intensity transformation in which the output pixel intensity is obtained by applying a logarithmic function to the input pixel intensity.
- It enhances low-intensity pixels while compressing high-intensity pixels, thereby improving the visibility of details in dark regions of an image.
- The mathematical expression for Log Transformation is

$$s = clog(1 + r)$$

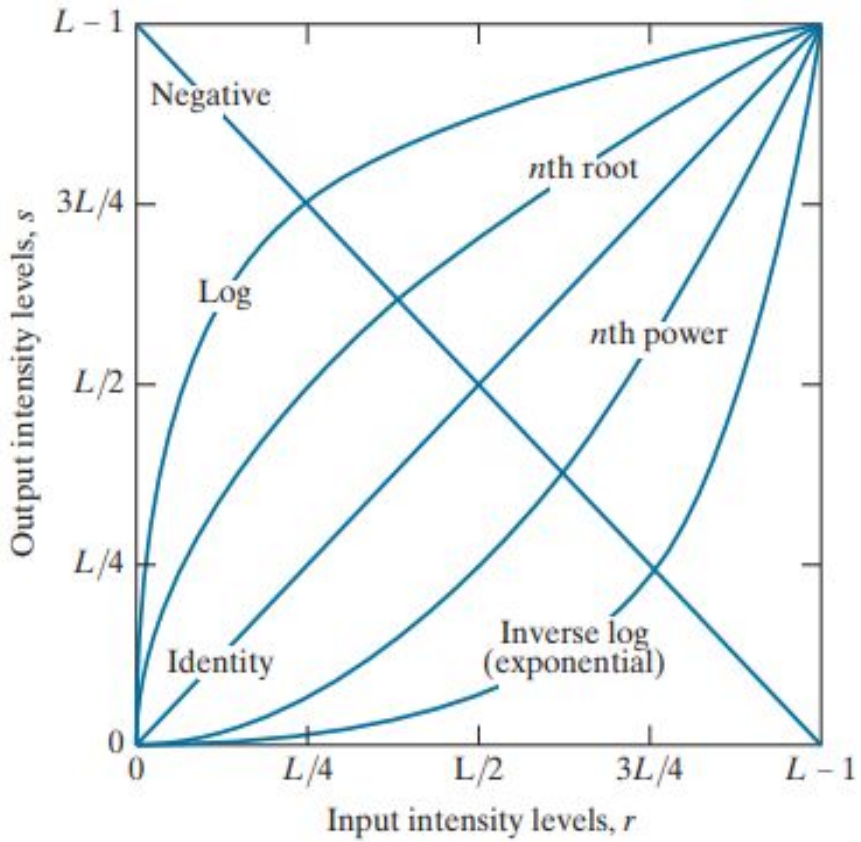
where, r = Input pixel intensity

s= Output pixel intensity

c= Scaling constant

- The addition of 1 ensures that the logarithm is defined even when the input intensity is zero. (if r = 0 the log(0) is undefined)

- The logarithmic function produces values over a relatively small range. Therefore, a scaling constant c is introduced to map the transformed intensities back to the full display range.
- For an 8-bit image,
- $c = \frac{255}{\log(1+255)}$
- This ensures that the maximum input intensity (255) is mapped close to the maximum output intensity (255).



Algorithm

1. Read the input image.
2. Determine the scaling constant ccc.
3. For each pixel, compute:

$$s = \mathrm{clog}(1 + r)$$

4. Replace the original intensity with the transformed intensity.
5. Display the transformed image.

Example

For Input pixel intensity r = 50
Scaling constant c=45

$$s = c\log(1+r)$$

$$s = 45\log(1+50)$$

$$s = 45\log(51)$$

$$\log(51) \approx 1.7076$$

$$s = 45 \times 1.7076$$

$$s \approx 76.84$$

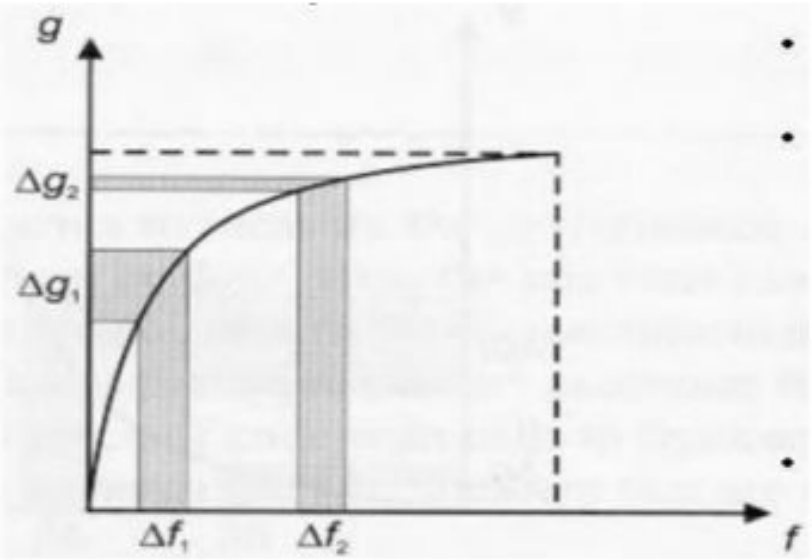


Fig. log transformation curve input vs outpu

For Input pixel intensity r = 200
Scaling constant c=45

$$r = 200$$

$$c = 45$$

$$s = 45\log(201)$$

$$\log(201) \approx 2.303$$

$$s = 45 \times 2.303$$

$$s \approx 103.6$$

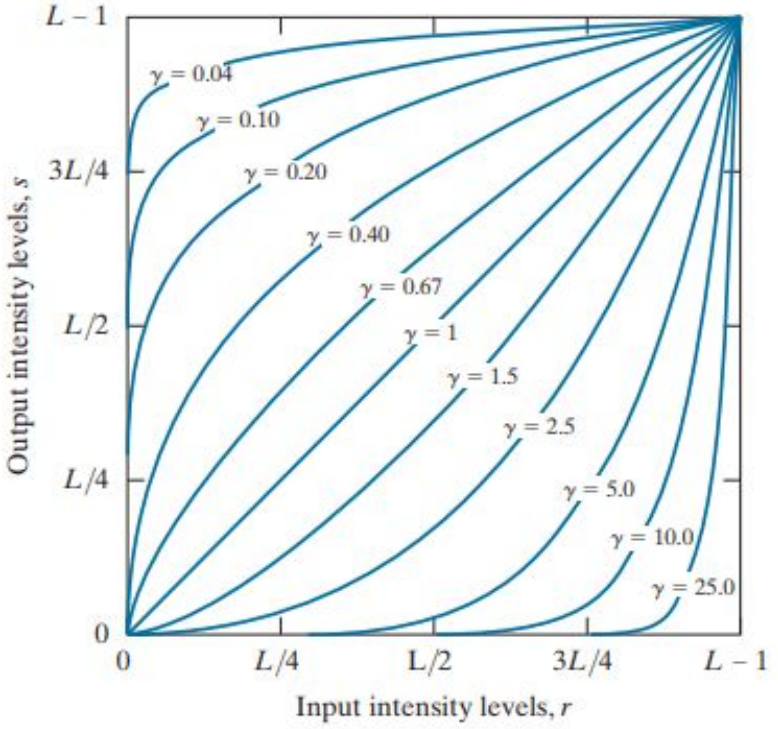
Power-Law Transformation

- Also known as **Gamma Transformation** or **Gamma Correction**
- Unlike Image Negative and Log Transformation, which have fixed mathematical relationships, Gamma Transformation provides flexible control over image brightness and contrast through a parameter called gamma (γ).
- Used in digital cameras, computer monitors, television displays, medical imaging, satellite imaging, photography, and computer graphics

- **Power-Law Transformation** is a nonlinear intensity transformation in which each pixel intensity is raised to a power (γ). The transformed intensity depends on the value of the gamma parameter.

$$s = c\, r^{\gamma}$$

Gamma Value	Effect on Image
γ = 0.3	Very bright
γ = 0.5	Bright
γ = 0.8	Slightly bright
γ = 1	No change
γ = 1.5	Slightly dark
γ = 2	Dark
γ = 3	Very dark



Why is Gamma Transformation Required?

- Digital images often appear too dark or too bright because:

  Camera sensors respond nonlinearly to light.

  Display devices (monitors, TVs, projectors) have nonlinear brightness characteristics.

  Human eyes perceive brightness nonlinearly.

- Gamma Transformation compensates for these nonlinearities and improves image appearance. It is required to:

  Correct brightness distortion.

  Improve image contrast.

  Reveal hidden details.

  Adjust image appearance for different display devices.

  Perform gamma correction in cameras and monitors.

Algorithm

Step 1: Read the input image

Step 2: Choose the gamma (γ) value.

Step 3: Normalize the pixel values to the range [0,1]:

$$r = 255I(i,j)$$

Step 4: Apply the Power-Law Transformation:

$$s = c\, r^{\gamma}$$

Step 5: Convert the normalized values back to the range [0,255]:

$$G(i,j) = 255 \times s$$

Step 6: Round the output values to the nearest integer.

Step 7: Repeat Steps 3–6 for every pixel in the image.

Step 8: Display the transformed image G.

Example

- For image, find Power law transformation
- Given c = 1, gamma = 0.5

$$I = \begin{matrix} 20 & 40 & 55 \\ 60 & 10 & 17 \\ 22 & 90 & 150 \end{matrix}$$

$$I = \begin{matrix} 71 & 101 & 119 \\ 124 & 50 & 66 \\ 75 & 151 & 196 \end{matrix}$$

Pixel	Norm = pixel/255	S = sqrt(0.5)	Output pixel
20	0.078	0.280	71
40	0.157	0.396	101
55	0.216	0.465	119
60	0.235	0.485	124
10	0.039	0.198	50
17	0.067	0.258	66
22	0.086	0.294	75
90	0.353	0.594	151
150	0.588	0.767	196

- **Why Do We Normalize?**

Suppose r=255 and γ=2

255^2 =65025

This value is far beyond the allowable 8-bit range (0–255).

Normalization keeps all input values between 0 and 1, preventing excessively large outputs.

Thresholding

- Thresholding is a process of converting a grayscale image into a **binary image** by comparing every pixel with a selected threshold value.
- If a pixel intensity is greater than or equal to the threshold, it is assigned one value (usually white). Otherwise, it is assigned another value (usually black).
- Thus, every pixel is classified into one of two categories:

  Foreground (Object)

  Background

- Thresholding is mathematically represented as

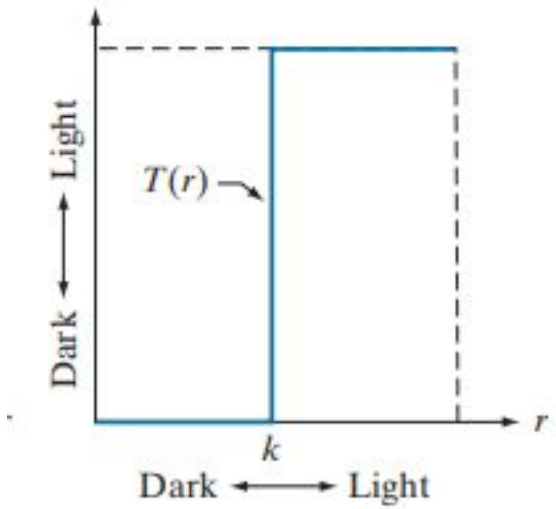
$$g(x) = \begin{cases} 255, f(x,y) \geq T0 \\ 0 \quad , f(x,y) < T0 \end{cases}$$

f(x, y) = Input image

g(x, y) = Output binary image

T = Threshold value

- This means that each pixel is compared with the threshold value independently.



Algorithm for Thresholding

- **Algorithm:**
- Read the grayscale image.
- Select the threshold value T.
- For each pixel (x,y)in the image:

$$If\ I(x,y) \geq T, set\ B(x,y) = 255$$
$$Otherwise, set\ B(x,y) = 0$$

- Repeat until all pixels are processed.
- Display or save the binary image.

Example

- Suppose we have an 8-bit grayscale image where pixel values range from 0 to 255.

  Dark pixels have lower intensity values.

  Bright pixels have higher intensity values.

- Now select a threshold value, say T = 120
- Then, Pixels having intensity greater than or equal to 120 become white (255).
- Pixels having intensity less than 120 become black (0).
- Thus, the grayscale image is converted into a binary image.

-

$$I = \begin{matrix} 71 & 101 & 119 \\ 124 & 50 & 66 \\ 75 & 151 & 196 \end{matrix}$$

At $T = 120$

$$I' = \begin{matrix} 0 & 0 & 0 \\ 255 & 0 & 0 \\ 0 & 255 & 255 \end{matrix}$$

Types of thresholding

- Global
- Adaptive

  In adaptive thresholding, the image is divided into small regions, and each region has its own threshold value.

  Leftside T=90 and Right side T=150

  This method works well when different parts of the image have different lighting conditions.

- OTSU
  - Otsu's method is an **automatic threshold selection technique**.
  - Instead of manually selecting the threshold, the algorithm analyses the image histogram and chooses the threshold that best separates the foreground and background by maximizing the separation between the two classes (equivalently, minimizing the variance within each class).

Contrast Stretching

- Contrast is the **difference in intensity (brightness)** between the darkest and brightest regions of an image.

  High contrast → Large difference between dark and bright pixels.

  Low contrast → Small difference between dark and bright pixels.

- Images with high contrast are sharp and easy to interpret, whereas low-contrast images appear dull, faded, or washed out.

- for example, an 8-bit grayscale image may use only values from 80 to 150, even though the available intensity range is 0 to 255.
- Contrast stretching redistributes these values across a larger range, making dark regions darker and bright regions brighter.
- The contrast stretching formula is

$$s = \frac{r - r_{min}}{r_{max} - r_{min}} \times (L - 1)$$

r = Original pixel value

s = New pixel value

r_{min}= Minimum intensity in the input image

r_{max} = Maximum intensity in the input image

L = Number of gray levels (256 for an 8-bit image)

Example

- Consider image

$$I = \begin{bmatrix} 80 & 100 & 120 \\ 140 & 160 & 180 \\ 90 & 110 & 150 \end{bmatrix}$$

$$r_{min} = 80$$

$$r_{max} = 180$$

$$s = \frac{r - 80}{100} \times 255$$

$$O = \begin{bmatrix} 0 & 51 & 102 \\ 153 & 204 & 255 \\ 26 & 77 & 179 \end{bmatrix}$$

Output image

General intensity-range transformation

Sometimes the desired output range is not 0 to 255. Suppose the input range is:

$$[r_{\min}, r_{\max}]$$

and the desired output range is:

$$[s_{\min}, s_{\max}]$$

Then:

$$s = s_{\min} + \frac{r - r_{\min}}{r_{\max} - r_{\min}}\left(s_{\max} - s_{\min}\right)$$

Example

Map the input range 50 –180 to the output range 20–220

Types of Contrast Stretching

1. Linear Contrast Stretching
2. **Piecewise Linear Contrast Stretching**

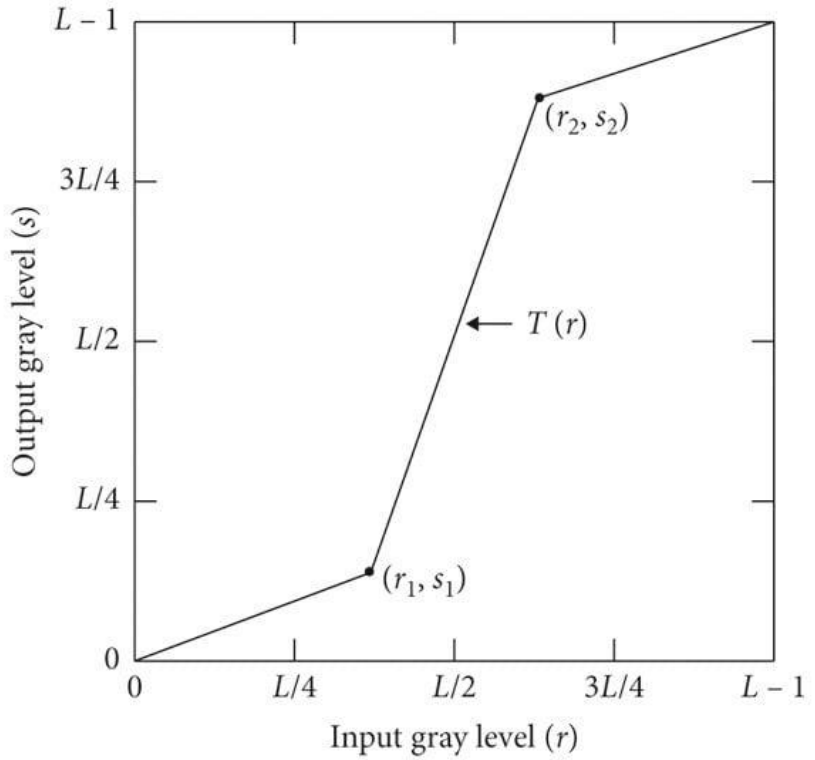
Simple linear stretching may not be appropriate when only a particular intensity region needs enhancement.

A piecewise-linear transformation uses

multiple line segments.

It has two control points, say (r1,s1) and (r2s2)

$$s = \begin{cases} \frac{s_1}{r_1} r, & 0 \le r \le r_1 \\ s_1 + \frac{s_2 - s_1}{r_2 - r_1}(r - r_1), & r_1 < r \le r_2 \\ s_2 + \frac{L-1-s_2}{L-1-r_2}(r - r_2), & r_2 < r \le L-1 \end{cases}$$



Example

- Suppose we want to enhance intensities between 80 and 160.
- Choose $(r1, s1) = (80,40) and\ (r2, s2) = (160, 220)$

$$m = \frac{220 - 40}{160 - 80}$$

$$m = \frac{180}{80} = 2.25$$

$$m > 1$$

- the contrast in the range 80–160 is strongly increased.
- Now for r = 120

$$s = 40 + \frac{220 - 40}{160 - 80}(120 - 8$$

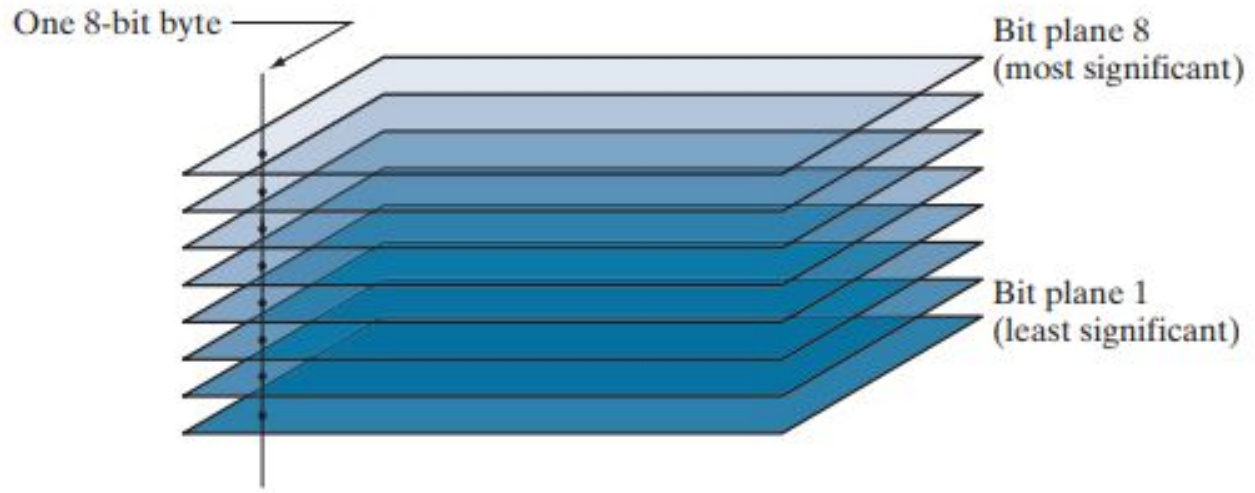
$$s = 40 + \frac{180}{80} \times 40$$

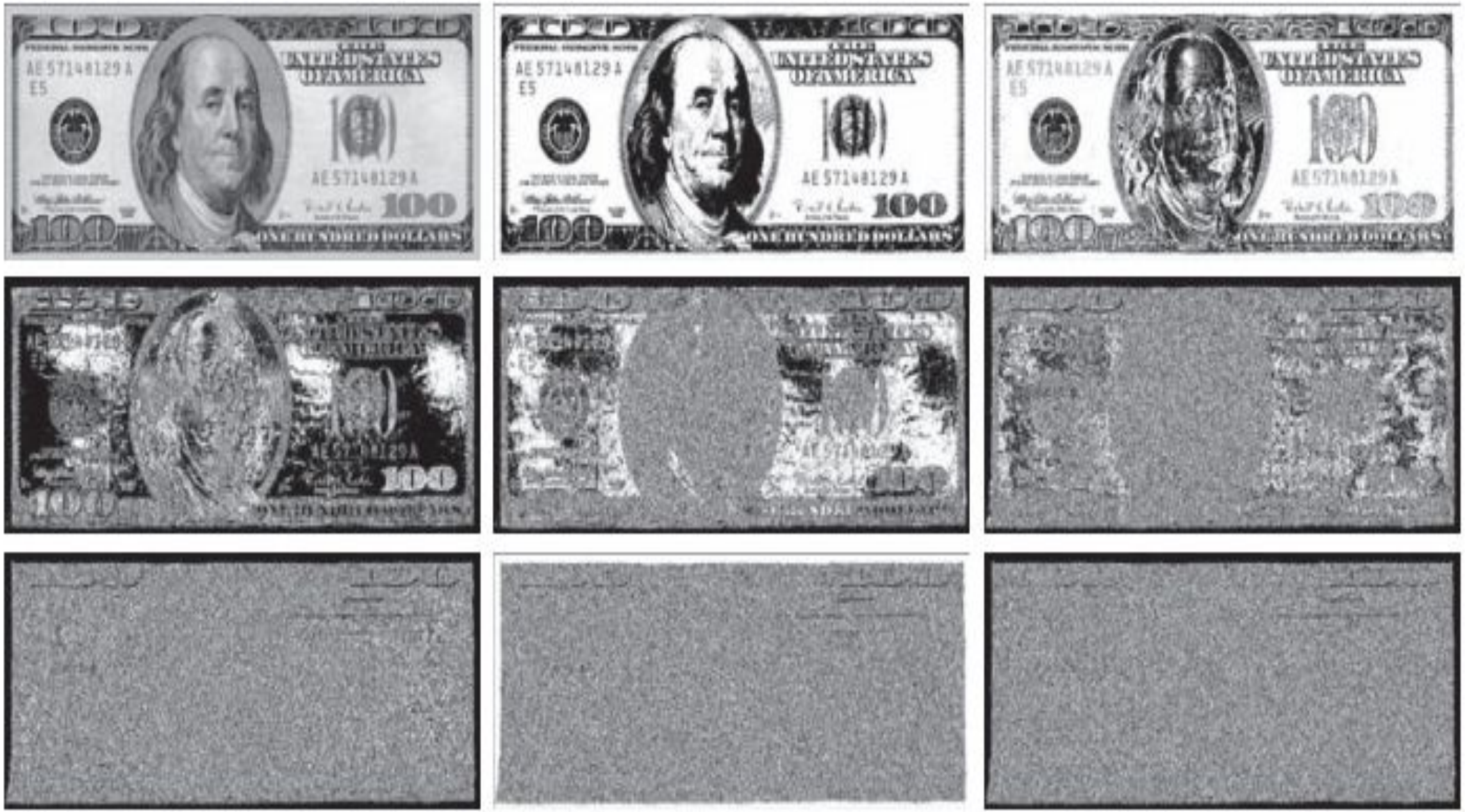
$$s = 40 + 90 = 130$$

$$120 \rightarrow 130$$

Bit plane slicing

- It is used to separate an image into different bit planes, allowing us to analyse how much information is contributed by each bit of every pixel.
- Unlike intensity transformations, which modify the gray-level values directly, bit plane slicing decomposes the binary representation of each pixel into individual layers (bit planes).





Example:

- Consider a small image

$$I = \begin{bmatrix} 201 & 140 \\ 95 & 60 \end{bmatrix}$$

- Convert each pixel to binary

201	11001001
140	10001100
95	01011111
60	00111100

Bit Plane 7

$$\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

Bit Plane 6

$$\begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$$

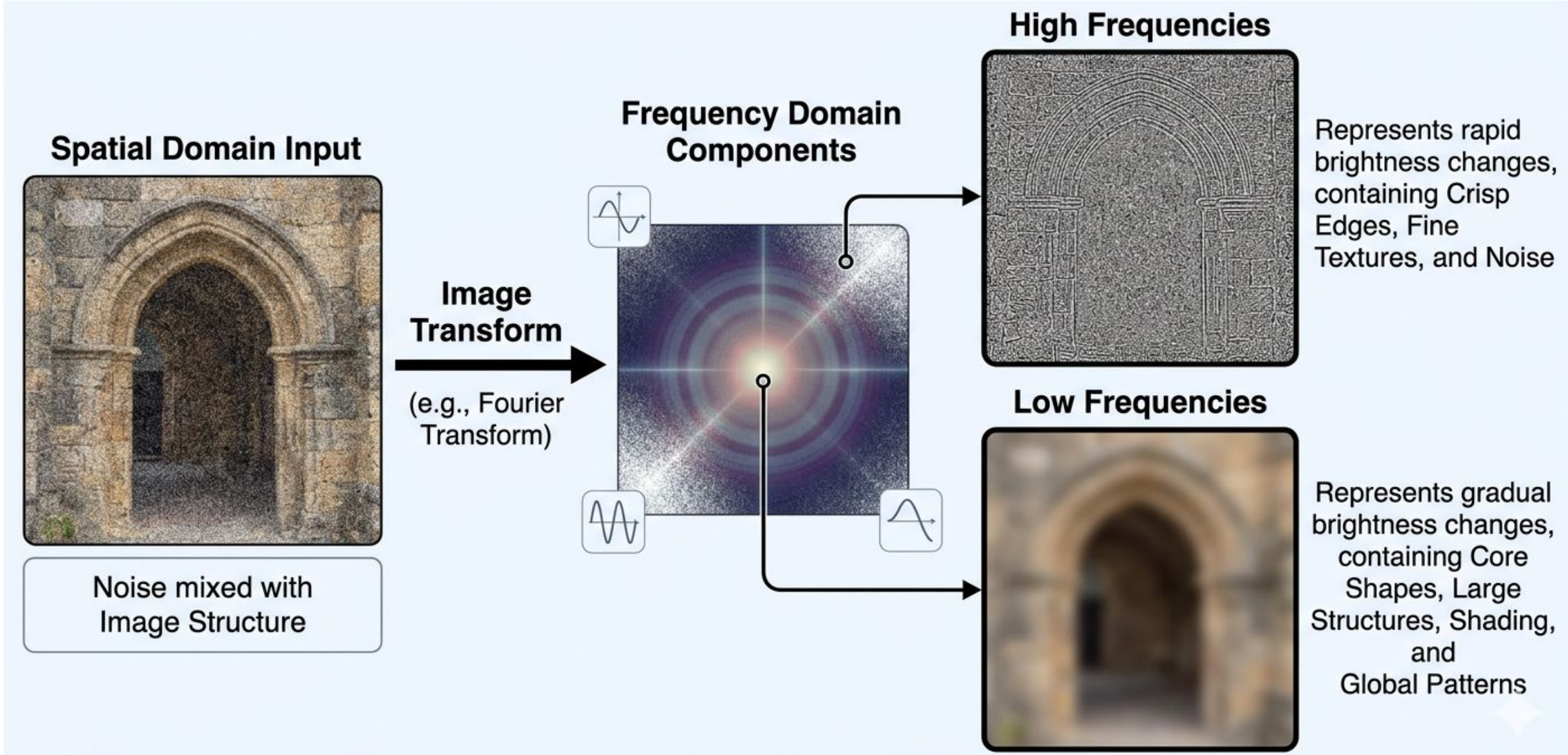
Bit Plane 5

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

IMAGE TRANSFORMS

- An **Image Transform** is a mathematical operation that converts an image from the **spatial domain** into another domain, known as the **transform domain** or **frequency domain.**
- It simplify image analysis, enhancement, compression, restoration, or feature extraction.
- Why Transforms?
  - Fast computation
  - Conceptual insights for various image processing
  - Obtain transformed data as measurement
  - For efficient storage and transmission

Example:


Spatial Domain Input
Noise mixed with Image Structure
Image Transform
(e.g., Fourier Transform)
Frequency Domain Components
High Frequencies
Represents rapid brightness changes, containing Crisp Edges, Fine Textures, and Noise
Low Frequencies
Represents gradual brightness changes, containing Core Shapes, Large Structures, Shading, and Global Patterns

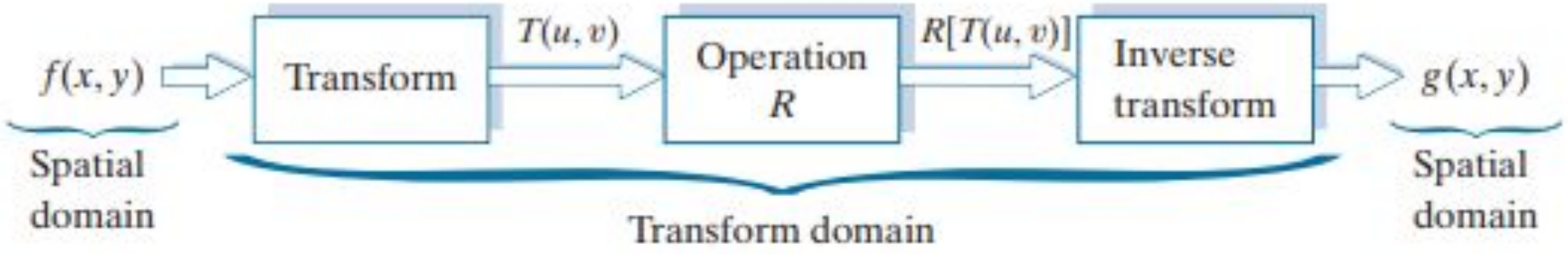
Example 2

JPEG compression.

- Instead of storing millions of pixels, JPEG stores only the important frequency coefficients using the Discrete Cosine Transform (DCT).
- Thus, image transforms simplify many image processing tasks.

Key steps:

- (1) Transform the image
- (2) Carry the task(s) in the transformed domain.
- (3) Apply inverse transform to return to the spatial domain.



- Forward Transformation

forward transformation kernel

$$T(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) r(x,y,u,v) \quad u = 0,1,\ldots,M-1, \; v = 0,1,\ldots,N-1$$

- Inverse Transformation

inverse transformation kernel

$$f(x,y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} T(u,v) s(x,y,u,v) \quad x = 0,1,\ldots,M-1, \; y = 0,1,\ldots,N-1$$

- A kernel is said to be separable if:

$$r(x, y, u, v) = r_1(x, u) r_2(y, v)$$

- A kernel is said to be symmetric if :

$$r(x, y, u, v) = r_1(x, u) r_1(y, v)$$

(i.e., r_1 and r_2 are functionally equal)

Types of Transforms

- Fourier Transform
  - Continuous FT
  - Discrete FT
  - Fast FT
- Discrete Cosine Transform (DCT)
- Wavelet Transform
- Hadamard Transform
- Walsh Transform
- Haar Transform etc

Fourier Transforms

- The **Continuous Fourier Transform (FT)** converts a continuous spatial-domain function into its continuous frequency-domain representation.
- It is mainly used in theoretical analysis and signal processing because digital computers work with discrete data.
- In DIP, the most commonly used transforms are **DFT** and **FFT**, because digital images are discrete.
- Continuous FT (1D)

$$\underline{\text{Forward FT:}}\quad F(f(x)) = F(u) = \int_{-\infty}^{\infty} f(x)e^{-j2\pi ux}\,dx$$

$$\underline{\text{Inverse FT:}}\quad F^{-1}(F(u)) = f(x) = \int_{-\infty}^{\infty} F(u)e^{j2\pi ux}\,du$$

where $e^{\pm j\theta} = cos(\theta) \pm jsin(\theta)$

Continuous Fourier Transform (2D)

$$F(u,v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x,y) e^{-j2\pi(ux+vy)} dx dy$$

where

- $f(x,y)$ = Spatial domain image
- $F(u,v)$ = Frequency spectrum
- u, v = Frequency variables

Task

- Develop a computer vision application that allows a user to improve the visual quality of an image and apply different geometrical transformations. The application should display the original and processed images side by side and allow comparison of different techniques.
- Mention all the required parameters.

- **Part A: Spatial-Domain Enhancement**
  - Implement the following point-processing operations:
    - Brightness adjustment
    - Contrast adjustment
    - Image negative
    - Log transformation
    - Power-law or gamma transformation
    - Thresholding
    - Contrast stretching

- **Part B: Geometrical Transformations**
  - Implement:
    - Translation
    - Scaling
    - Rotation
    - Horizontal reflection
    - Vertical reflection
    - Reflection about the origin
    - Shearing along the x-axis
    - Shearing along the y-axis
    - Affine transformation